LAPORAN PRAKTIKUM

SISKA



Amos Belsing Arta Baringbing

43324046

Kelompok 07

D-III Teknologi Komputer

INSTITUT TEKNOLOGI DEL

FAKULTAS VOKASI

Parameter Dokumentasi	Rincian Spesifikasi Lingkungan Praktikum
Platform Virtualisasi Host	VMware Workstation Pro (Nested Virtualization VT-x/AMD-V - VM: Ubuntu Amos)
Sistem Operasi Host Utama	Ubuntu Server 24.04 LTS (Noble Numbat) - Hostname: amosb
Pengguna Administrasi Cloud	stack (Sudoers NOPASSWD) pada direktori /home/stack/devstack
Distribusi Cloud Platform	OpenStack DevStack (Keystone, Glance, Nova, Neutron, Cinder, Horizon)
Jaringan Eksternal (Floating)	Subnet 172.24.4.0/24 dengan Gateway Host 172.24.4.1
Jaringan Internal (Private)	Subnet 10.0.0.0/24 dengan Router Virtual Neutron (OVN/OVS)
Instance Komputasi Aktif	server-pertama & server-kedua (Ubuntu 22.04 Jammy - IP: 172.24.4.30)
Layanan Basis Data Instance	MariaDB Server 10.6.23 (Port 3306, Bind 0.0.0.0, DB: database_anda)
Layanan Proxy Jaringan Host	Tinyproxy 1.11.1 (Port 8888 pada 172.24.4.1 untuk Bypass MTU)
Standar Tipografi Dokumen	Times New Roman 12pt, Spasi Baris 1.5, Dilengkapi Screenshot Tahapan

DAFTAR ISI DOKUMEN (HALAMAN 1 - 34)

Dokumen edisi kedua (v2) ini dilengkapi dengan tangkapan layar (screenshot) langkah demi langkah mulai dari penyiapan mesin virtual Ubuntu Amos di VMware Workstation, instalasi DevStack, hingga orkestrasi instance dan basis data MariaDB.

Bab / Subbab	Judul Materi Pembahasan Langkah Demi Langkah	Hal.
Halaman Sampul	Identitas Modul dan Spesifikasi Lingkungan Sistem Cloud	1
Daftar Isi	Pemetaan Struktur Bab, Daftar Gambar Screenshot, dan Konfigurasi	2 - 4
BAB 1	PENDAHULUAN DAN ARSITEKTUR CLOUD OPENSTACK DEVSTACK	5 - 8
1.1 - 1.2	Konsep IaaS Private Cloud dan Komponen Inti OpenStack	5 - 6
1.3 - 1.4	Topologi Nested Virtualization VMware dan Alokasi Sumber Daya	7 - 8
BAB 2	PERSIAPAN VMWARE WORKSTATION DAN INSTALASI UBUNTU HOST	9 - 12
2.1 - 2.2	Konfigurasi VM Ubuntu Amos (Gambar 2.1) & Hardware Virtualization	9 - 10
2.3 - 2.4	Instalasi Ubuntu Server, Partisi LVM, dan Verifikasi OpenSSH	11 - 12
BAB 3	KONFIGURASI SISTEM OPERASI HOST DAN PENGGUNA STACK	13 - 16
3.1 - 3.2	IP Statis Netplan & Pembuatan Akun Sudoers stack (Gambar 3.1)	13 - 14
3.3 - 3.4	Instalasi Dependensi Dasar dan Tuning Kernel sysctl.conf	15 - 16
BAB 4	KLONING REPOSITORI DEVSTACK DAN	17 - 20

	PENYUSUNAN LOCAL.CONF	
4.1 - 4.2	Pengunduhan Git DevStack dan Bedah Struktur Skrip Utilitas	17 - 18
4.3 - 4.4	Penyusunan Berkas local.conf (Gambar 4.1) & Analisis Parameter	19 - 20
BAB 5	EKSEKUSI INSTALASI DEVSTACK (STACK.SH) DAN VERIFIKASI	21 - 25
5.1 - 5.2	Proses Eksekusi ./stack.sh & Keluaran Sukses (Gambar 5.1)	21 - 22
5.3 - 5.5	Verifikasi Unit Systemd, Horizon Web UI, dan Berkas openrc	23 - 25
BAB 6	MANAJEMEN IMAGE GLANCE DAN KONFIGURASI FLAVOR NOVA	26 - 29
6.1 - 6.2	Arsitektur Glance & Unggah Image Ubuntu 22.04 (Gambar 6.1)	26 - 27
6.3 - 6.4	Inspeksi Metadata Image QCOW2 dan Pembuatan Custom Flavor	28 - 29
BAB 7	KONFIGURASI JARINGAN NEUTRON, ROUTER, DAN SECURITY GROUP	30 - 34
7.1 - 7.3	Jaringan Public 172.24.4.0/24 (Gambar 7.1), Private, & Router	30 - 32
7.4 - 7.5	Manajemen SSH Keypair dan Aturan Dasar Security Group	33 - 34

DAFTAR ISI DOKUMEN (HALAMAN 35 - 60)

Bagian kedua memuat dokumentasi visual lengkap saat meluncurkan instance server-kedua, mengatasi kemacetan unduhan APT (Path MTU Black Hole), dialog ungu needrestart, instalasi MariaDB, hingga pemulihan konflik layanan MySQL pada host DevStack.

Bab / Subbab	Judul Materi Pembahasan Langkah Demi Langkah	Hal.
BAB 8	PEMBUATAN DAN PELUNCURAN INSTANCE KOMPUTASI UBUNTU	35 - 39
8.1 - 8.2	Prosedur Peluncuran server-pertama dan server-kedua via CLI/GUI	35 - 36
8.3 - 8.5	Asosiasi Floating IP 172.24.4.30 & Login SSH (Gambar 8.1)	37 - 39
BAB 9	DIAGNOSIS KENDALA PATH MTU BLACK HOLE PADA INSTANCE	40 - 44
9.1 - 9.2	Bukti Screenshot Kemacetan APT di 27.5 kB / 21% (Gambar 9.1)	40 - 41
9.3 - 9.5	Screenshot Eksperimen MTU 1200 & Mangle TCPMSS 1160 (Gambar 9.2)	42 - 44
BAB 10	IMPLEMENTASI TINYPROXY HOST SEBAGAI SOLUSI APT INSTANCE	45 - 48
10.1 - 10.2	Instalasi Tinyproxy Host & Konfigurasi 01proxy (Gambar 10.1)	45 - 46
10.3 - 10.4	Screenshot Dialog Ungu needrestart Ubuntu 22.04 (Gambar 10.2)	47 - 48
BAB 11	INSTALASI DAN PENGAMANAN MARIADB DI SERVER-KEDUA	49 - 52
11.1 - 11.2	Screenshot Instalasi MariaDB (Gambar 11.1) & Status (Gambar	49 - 50

	11.2)	
11.3 - 11.4	Screenshot mysql_secure_installation (Gambar 11.3) & Bind 0.0.0.0	51 - 52
BAB 12	MANAJEMEN BASIS DATA, PENGGUNA, DAN RESOLUSI ERROR 1045	53 - 55
12.1 - 12.2	Screenshot Pembuatan DB (Gambar 12.1) & Resolusi Error 1045 (12.2-12.3)	53 - 54
12.3	Screenshot Pembuatan Tabel pengguna & Data Amos (Gambar 12.4-12.5)	55
BAB 13	PEMULIHAN KONFLIK PAKET MYSQL HOST DAN RESOLUSI HTTP 500	56 - 58
13.1 - 13.2	Screenshot Insiden HTTP 500 & Log APT History (Gambar 13.1-13.5)	56 - 57
13.3	Purge mariadb-client di Host dan Reinstalasi mysql-server-8.0	58
BAB 14	KONFIGURASI SECURITY GROUP PORT 3306 & UJI KONEKSI REMOTE	59 - 60
14.1 - 14.2	Pembukaan Port 3306 & Screenshot Kueri Jarak Jauh (Gambar 14.1)	59 - 60

DAFTAR GAMBAR TANGKAPAN LAYAR (SCREENSHOT) PRAKTIKUM

Tabel di bawah ini memuat daftar seluruh gambar tangkapan layar (screenshot) asli dari sesi praktikum dan terminal eksekusi yang disematkan pada masing-masing bab di dalam dokumen ini.

Nomor Gambar	Deskripsi Tangkapan Layar (Screenshot) di Dalam Dokumen	Hal.
Gambar 2.1	Antarmuka VMware Workstation dengan Mesin Virtual Ubuntu Amos	9
Gambar 3.1	Verifikasi Akselerasi KVM (/dev/kvm) dan Konfigurasi Sudoers stack	14
Gambar 4.1	Kloning Repositori Git DevStack dan Isi Berkas local.conf	19
Gambar 5.1	Keluaran Akhir Eksekusi ./stack.sh dan Aktivasi Kredensial openrc	22
Gambar 6.1	Unggah dan Verifikasi Citra Ubuntu-22.04 Cloud Image pada Glance	27
Gambar 7.1	Daftar Jaringan Neutron (private & public) dan Bridge br-ex Host	30
Gambar 8.1	Asosiasi Floating IP 172.24.4.30 ke server-kedua dan Login SSH	37
Gambar 9.1	Kemacetan Unduhan sudo apt update pada 27.5 kB (21%) di server-kedua	40
Gambar 9.2	Penurunan MTU ens3 ke 1200 dan Tabel Mangle TCPMSS set 1160	42
Gambar 10.1	Instalasi Tinyproxy di Host DevStack dan Konfigurasi 01proxy APT	46
Gambar 10.2	Kotak Dialog Ungu needrestart (Daemons using outdated libraries)	47
Gambar 11.1	Proses Instalasi Paket mariadb-server-10.6 pada server-kedua	49
Gambar 11.2	Status Active (Running) Layanan	50

	MariaDB pada Systemctl	
Gambar 11.3	Eksekusi Skrip Pengamanan sudo mysql_secure_installation	51
Gambar 12.1	Konfigurasi bind-address 0.0.0.0, Pembuatan database_anda & admin_db	53
Gambar 12.2 & 12.3	Verifikasi Port 0.0.0.0:3306 dan Penanganan Galat ERROR 1045	54
Gambar 12.4 & 12.5	Pembuatan Tabel pengguna (Data Amos) dan Uji Kueri Lokal/Remote	55
Gambar 13.1 & 13.2	Konflik Security Group default dan Munculnya HTTP 500 di Host	56
Gambar 13.3 - 13.5	Matinya mysql.service Host dan Bukti Log /var/log/apt/history.log	57
Gambar 14.1	Pemulihan MySQL Host, Aturan Port 3306, dan Kueri Jarak Jauh Sukses	60

BAB 1: PENDAHULUAN DAN ARSITEKTUR CLOUD OPENSTACK DEVSTACK

1.1 Konsep Infrastructure as a Service (IaaS) dan OpenStack

Infrastructure as a Service (IaaS) merupakan model layanan komputasi awan yang menyediakan sumber daya infrastruktur teknologi informasi mendasar seperti kapasitas komputasi (CPU dan RAM), ruang penyimpanan blok maupun objek, serta perangkat jaringan virtual secara terprogram. Dalam arsitektur IaaS, administrator dan pengembang tidak perlu mengelola perangkat keras fisik secara manual setiap kali hendak meluncurkan peladen baru, melainkan cukup memanggil antarmuka pemrograman aplikasi (API) atau menggunakan antarmuka baris perintah (CLI).

OpenStack hadir sebagai platform perangkat lunak sumber terbuka (open-source) terkemuka untuk membangun dan mengelola infrastruktur cloud publik maupun privat. OpenStack dirancang secara modular, di mana setiap fungsi infrastruktur ditangani oleh komponen layanan spesifik yang saling berkomunikasi melalui antarmuka RESTful API dan antrean pesan terdistribusi (RabbitMQ) serta menyimpan status operasionalnya pada sistem manajemen basis data relasional (MySQL atau MariaDB).

Dalam kegiatan laboratorium dan pengembangan arsitektur sistem terdistribusi, penginstalan seluruh komponen OpenStack secara manual di banyak peladen fisik memerlukan sumber daya yang sangat besar. Oleh sebab itu, komunitas OpenStack mengembangkan DevStack, yaitu sekumpulan skrip otomatisasi berbasis Bash yang mampu memasang, mengonfigurasi, dan mengintegrasikan seluruh layanan inti OpenStack di dalam satu mesin tunggal (All-in-One Node). Melalui pendekatan DevStack, seluruh siklus hidup penyediaan infrastruktur mulai dari manajemen identitas, unggah citra sistem operasi, pembuatan jaringan virtual, peluncuran instance, hingga implementasi peladen basis data dapat dipraktikkan secara utuh dan presisi.

- **Otomatisasi Penuh:** Skrip stack.sh mengonfigurasi repositori, layanan database, message broker, dan hypervisor KVM/QEMU secara terpadu.
- **Isolasi Jaringan Virtual:** Mendukung pembuatan jaringan privat, router virtual, Network Address Translation (NAT), dan Floating IP.
- **Reprodusibilitas Tinggi:** Seluruh parameter lingkungan didefinisikan dalam satu berkas konfigurasi terpusat yaitu local.conf.

1.2 Arsitektur dan Interaksi Komponen Inti OpenStack

Keberhasilan pengoperasian OpenStack bergantung pada sinkronisasi kerja antar-komponen intinya. Apabila salah satu komponen mengalami gangguan atau kehilangan koneksi ke basis data utama host, maka rantai penyediaan layanan akan terputus dan menghasilkan kode kesalahan HTTP 500 (Internal Server Error). Tabel berikut menjabarkan peran teknis masing-masing layanan inti yang berjalan di dalam lingkungan DevStack.

Nama Layanan	Nama Proyek	Fungsi dan Tanggung Jawab Teknis dalam Arsitektur Cloud
Identity Service	Keystone	Menangani autentikasi pengguna, validasi token akses, katalog endpoint API, serta manajemen proyek (tenant) dan peran (role).
Compute Service	Nova	Mengelola siklus hidup instance mesin virtual di atas hypervisor Libvirt/KVM, penjadwalan CPU/RAM, dan pemasangan antarmuka jaringan.
Networking Service	Neutron	Menyediakan Software-Defined Networking (SDN), manajemen subnet, router virtual, DHCP agent, Floating IP, dan aturan Security Group.
Image Service	Glance	Menyimpan, mendaftarkan, dan mendistribusikan citra disk sistem operasi (seperti Cirros dan Ubuntu Cloud Image berformat QCOW2).
Block Storage	Cinder	Menyediakan volume penyimpanan blok persisten berbasis LVM/iSCSI yang dapat dipasang (attach) ke instance komputasi.
Placement API	Placement	Melacak inventaris dan pemakaian sumber daya perangkat keras (vCPU, memori, dan ruang disk) pada setiap node komputasi.
Dashboard UI	Horizon	Menyediakan antarmuka grafis berbasis web (Django) di atas

Apache HTTP Server untuk administrasi sumber daya cloud.

Seluruh komponen di atas menyimpan tabel konfigurasinya di dalam instance MySQL Server yang berjalan pada host DevStack. Saat seorang administrator mengeksekusi perintah pembuatan instance atau pembukaan port Security Group melalui CLI, perintah tersebut terlebih dahulu diverifikasi oleh Keystone, lalu diteruskan ke Nova atau Neutron yang kemudian melakukan pembacaan serta penulisan catatan transaksi pada basis data MySQL host.

1.3 Topologi Jaringan Virtualisasi Bertingkat (Nested Virtualization)

Praktikum ini diimplementasikan menggunakan arsitektur virtualisasi bertingkat (nested virtualization). Lapisan pertama (Level 0) adalah sistem operasi utama Windows pada perangkat fisik yang menjalankan hypervisor tipe-2 VMware Workstation Pro. Di atas VMware Workstation, dijalankan mesin virtual lapisan kedua (Level 1) bernama Ubuntu Amos (hostname: amosb) yang berfungsi sebagai Host DevStack sekaligus hypervisor KVM/QEMU bagi instance OpenStack.

Diagram Alir Enkapsulasi Paket pada Nested Virtualization

```
[ Sistem Operasi Host Fisik (Windows) ]
       |
       +-- [ VMware Virtual Network (VMnet8 NAT / Bridged MTU 1500) ]
              |
              +-- [ Host DevStack: Ubuntu Amos (stack@amosb) ]
                     |-- Interface Fisik VM: ens33
                     |-- Bridge Eksternal Neutron: br-ex (172.24.4.1/24)
                     |-- Layanan Proxy Host: Tinyproxy (172.24.4.1:8888)
                     |
                     +-- [ Jaringan Overlay OVN/Geneve & Router Virtual Neutron ]
                            |
                            +-- [ Instance Lapisan 2: server-kedua (Ubuntu 22.04)
]
                                   |-- Interface Virtual: ens3 (Private IP
10.0.0.x)
                                   |-- Floating IP NAT: 172.24.4.30
                                   |-- Layanan Aktif: MariaDB Server
(0.0.0.0:3306)
```

Konsekuensi teknis paling krusial dari arsitektur bertingkat ini terletak pada penambahan header enkapsulasi jaringan (Geneve atau VXLAN sebesar 50 hingga 58 byte) oleh OpenStack Neutron. Ketika paket data dari dalam instance server-kedua keluar menuju internet melalui router virtual Neutron dan jembatan VMware, ukuran paket penuh (1500 byte) akan melebihi batas Maximum Transmission Unit (MTU) antarmuka virtual. Pemahaman terhadap topologi ini menjadi kunci utama ketika kita menyelesaikan kendala kemacetan unduhan paket APT pada bab selanjutnya.

1.4 Spesifikasi Sumber Daya Host dan Instance Praktikum

Agar seluruh layanan OpenStack DevStack beserta dua instance Ubuntu 22.04 LTS dapat berjalan secara simultan tanpa mengalami kehabisan memori (Out-of-Memory Killer) maupun kemacetan antrean instruksi prosesor, alokasi sumber daya harus direncanakan secara proporsional. Berikut adalah rincian alokasi perangkat keras virtual dan perangkat lunak yang digunakan sepanjang siklus implementasi.

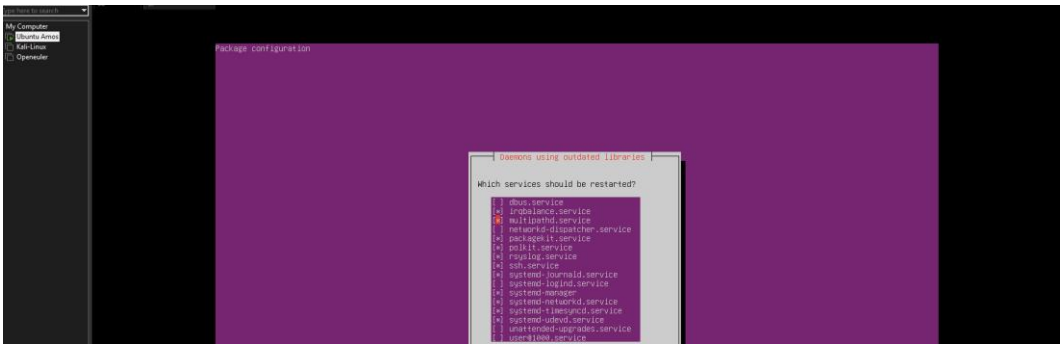
Entitas Mesin	Komponen Sumber Daya	Nilai Konfigurasi dan Keterangan Teknis
Host DevStack (amosb)	Prosesor (vCPU)	Minimal 4 Core (Disarankan 6 - 8 Thread) dengan fitur Intel VT-x / AMD-V aktif
Host DevStack (amosb)	Memori Utama (RAM)	Minimal 8 GB (Disarankan 12 GB - 16 GB) ditambah ruang Swap aktif
Host DevStack (amosb)	Kapasitas Penyimpanan	80 GB - 100 GB Virtual Disk (SCSI/NVMe) untuk citra OS dan volume LVM
Host DevStack (amosb)	Basis Data Internal	MySQL Community Server 8.0.46 (Ubuntu 24.04) pada port lokal 3306
Instance: server-pertama	Peran Fungsional	Instance Komputasi Pertama / Pengujian Konektivitas Jaringan Antar-VM
Instance: server-kedua	Peran Fungsional	Dedicated Database Server (MariaDB 10.6.23) untuk penyimpanan aplikasi
Instance: server-kedua	Citra Sistem Operasi	Ubuntu Server 22.04 LTS (Jammy Jellyfish) Cloud Image AMD64
Instance: server-kedua	Flavor Komputasi	1 - 2 vCPU, 1024 - 2048 MB RAM, 10 - 20 GB Root Disk
Instance: server-kedua	Identitas Jaringan	Hostname: server-kedua, User: ubuntu, Floating IP: 172.24.4.30

Seluruh tahapan pada bab-bab selanjutnya mengacu secara konsisten pada parameter di atas. Nama pengguna host adalah stack pada mesin amosb, sedangkan nama pengguna di dalam instance adalah ubuntu pada mesin server-kedua.

BAB 2: PERSIAPAN VMWARE WORKSTATION DAN INSTALASI UBUNTU HOST

2.1 Pembuatan Mesin Virtual Host dan Aktivasi Nested Virtualization

Tahap pertama dalam membangun laboratorium OpenStack DevStack adalah menyiapkan mesin virtual host bernama Ubuntu Amos pada perangkat lunak VMware Workstation. Karena host ini nantinya akan menjalankan mesin virtual lagi di dalamnya (instance OpenStack), maka fitur penerusan instruksi virtualisasi perangkat keras (Hardware-Assisted Virtualization) wajib diaktifkan pada pengaturan prosesor VMware.



Gambar 2.1: Antarmuka VMware Workstation dengan Mesin Virtual 'Ubuntu Amos' pada Panel My Computer

- **Langkah 1 (Pembuatan VM Baru):** Buka VMware Workstation, pilih menu File -> New Virtual Machine, pilih Typical, dan arahkan ke berkas ISO Ubuntu Server 24.04 LTS.
- **Langkah 2 (Penamaan Mesin Virtual):** Beri nama mesin virtual sebagai Ubuntu Amos (terlihat pada panel kiri My Computer bersama Kali-Linux dan Openeuler pada Gambar 2.1).
- **Langkah 3 (Aktivasi VT-x/AMD-V):** Pada menu Customize Hardware -> Processors, centang opsi Virtualize Intel VT-x/EPT or AMD-V/RVI agar hypervisor KVM (/dev/kvm) aktif di dalam host.

2.2 Langkah Demi Langkah Instalasi Sistem Operasi Ubuntu Server

Setelah konfigurasi perangkat keras mesin virtual Ubuntu Amos selesai disimpan, nyalakan mesin virtual dengan menekan tombol Power on this virtual machine. Proses booting akan memuat pemasang berbasis teks (Subiquity Installer) milik Ubuntu Server. Ikuti urutan langkah konfigurasi berikut secara berurutan.

- **1. Pemilihan Bahasa Pemasang:** Pilih English menggunakan tombol panah atas/bawah, kemudian tekan Enter.
- **2. Pembaruan Installer:** Jika muncul opsi Update to the new installer, pilih Continue without updating agar proses instalasi berlangsung cepat.
- **3. Tata Letak Papan Ketik:** Biarkan nilai Layout dan Variant pada pengaturan bawaan English (US), lalu arahkan kursor ke tombol Done dan tekan Enter.
- **4. Jenis Instalasi:** Pilih opsi standar Ubuntu Server (bukan minimized) agar perkakas administrasi sistem dasar langsung terpasang.
- **5. Konfigurasi Antarmuka Jaringan:** Pastikan antarmuka utama (misalnya ens33) memperoleh alamat IPv4 secara otomatis dari layanan DHCP VMware NAT, lalu pilih Done.
- **6. Konfigurasi Proxy dan Mirror Awal:** Kosongkan kolom Proxy address pada tahap instalasi awal, dan gunakan alamat repositori mirror bawaan Ubuntu.

Ringkasan Parameter Identitas Sistem Host (Profile Setup)

```
Your name            : amosb
Your server's name   : amosb
Pick a username      : amosb
Choose a password    : [Masukkan kata sandi administrasi host]
Confirm your password: [Ketik ulang kata sandi yang sama]
```

Pastikan Anda mencatat kata sandi pengguna awal ini dengan baik karena akun tersebut akan digunakan pada saat login pertama kali untuk membuat pengguna khusus stack serta mengatur hak akses root.

2.3 Konfigurasi Penyimpanan LVM dan Pemasangan OpenSSH Server

Pada tahap konfigurasi penyimpanan (Guided storage configuration), biarkan opsi Use an entire disk dan Set up this disk as an LVM group tetap tercentang. Secara bawaan, pemasang Ubuntu Server terkadang hanya mengalokasikan sebagian kapasitas Logical Volume (LV) untuk partisi root (/). Oleh karena itu, sangat penting untuk memeriksa layar ringkasan penyimpanan (Storage configuration summary) dan memaksimalkan kapasitas partisi root.

Pengaturan Kapasitas Logical Volume (ubuntu-lv)

```
MOUNT POINT   SIZE       TYPE    DEVICE
/             78.000G    ext4    ubuntu-vg/ubuntu-lv   <-- Pastikan kapasitas
maksimal
/boot         2.000G     ext4    /dev/sda2

Langkah koreksi jika ukuran ubuntu-lv masih kecil:
1. Pilih baris ubuntu-lv lalu tekan Enter -> pilih Edit.
2. Ubah nilai Size menjadi nilai maksimal yang tersedia, lalu pilih Save.
3. Pilih Done dan konfirmasi dengan memilih Continue pada jendela peringatan.
```

Setelah melewati tahap pengaturan nama pengguna pada layar Profile setup, pemasang akan menampilkan layar SSH Setup. Tekan tombol Spasi pada opsi Install OpenSSH server hingga muncul tanda silang [X], lalu pilih Done. Pada layar Featured Server Snaps, jangan mencentang paket snap apa pun agar tidak terjadi tumpang tindih layanan dengan paket yang akan dipasang oleh DevStack. Tunggu hingga proses penyalinan sistem selesai, lalu pilih Reboot Now.

- **Pelepasan Media ISO:** Jika saat reboot muncul pesan Please remove the installation medium, cukup tekan tombol Enter karena VMware otomatis melepaskan kaitan ISO.
- **Pengujian Login Konsol:** Setelah layar login amosb login: muncul, masuklah menggunakan akun yang telah dibuat dan pastikan sistem berjalan lancar.

2.4 Verifikasi Lingkungan Host, Akselerasi KVM, dan Koneksi SSH

Begitu berhasil masuk ke dalam sesi terminal Ubuntu Server pada mesin host amosb, langkah verifikasi pertama yang wajib dilakukan adalah memeriksa ketersediaan modul virtualisasi perangkat keras KVM, memeriksa alamat IP antarmuka jaringan, serta memastikan sistem operasi memiliki paket dasar terbaru.

Perintah Verifikasi Awal pada Terminal Host (amosb)

```
# 1. Periksa ketersediaan instruksi virtualisasi CPU (harus lebih besar dari 0)
egrep -c '(vmx|svm)' /proc/cpuinfo

# 2. Periksa keberadaan berkas perangkat KVM
ls -la /dev/kvm

# 3. Periksa alamat IP host dan rute gateway internet
ip -br addr show
ip route show

# 4. Uji koneksi ke internet dan perbarui daftar paket dasar
ping -c 3 8.8.8.8
sudo apt update && sudo apt upgrade -y
```

Apabila perintah egrep menghasilkan angka 4 (atau sesuai jumlah core yang dialokasikan) dan berkas /dev/kvm terdeteksi, berarti fitur Nested Virtualization telah bekerja dengan sempurna. Selanjutnya, Anda dapat menggunakan aplikasi klien SSH dari sistem operasi Windows untuk terhubung ke alamat IP host amosb sehingga penyalinan perintah konfigurasi DevStack menjadi jauh lebih mudah.

BAB 3: KONFIGURASI SISTEM OPERASI HOST DAN PENGGUNA STACK

3.1 Penetapan Alamat IP Statis Host Menggunakan Netplan

Layanan OpenStack mengikat (bind) seluruh titik akhir (endpoint) REST API Keystone, Nova, Neutron, dan Glance pada alamat IP host yang ditetapkan saat instalasi. Apabila alamat IP host berubah akibat pembaruan masa sewa (lease) DHCP dari VMware, maka seluruh layanan OpenStack akan gagal berkomunikasi. Oleh sebab itu, antarmuka jaringan utama host wajib dikonfigurasi agar menggunakan alamat IP statis atau reservasi IP tetap.

Contoh Konfigurasi Netplan (/etc/netplan/00-installer-config.yaml)

```
network:
  ethernets:
    ens33:
      dhcp4: no
      addresses:
        - 192.168.100.50/24
      routes:
        - to: default
          via: 192.168.100.2
      nameservers:
        addresses:
          - 8.8.8.8
          - 1.1.1.1
  version: 2
```

Untuk menerapkan konfigurasi Netplan di atas, sesuaikan nama antarmuka (misalnya ens33) dan subnet dengan rentang jaringan VMnet8 VMware Anda, lalu jalankan perintah sudo netplan apply. Selain itu, pastikan berkas /etc/hosts memuat pemetaan nama host amosb ke alamat IP lokal agar resolusi nama host internal berjalan instan tanpa penundaan kueri DNS.

- **Verifikasi Hostname:** Jalankan perintah hostnamectl untuk memastikan Static hostname bernilai amosb.
- **Verifikasi DNS:** Jalankan perintah resolvectl status untuk memastikan server DNS aktif merespons permintaan nama domain.

3.2 Pembuatan Pengguna Khusus stack dan Konfigurasi Sudo Tanpa Sandi

Skrip instalasi DevStack menolak dijalankan secara langsung oleh pengguna root dan mengharuskan adanya akun non-root bernama stack yang memiliki hak akses sudo tanpa kata sandi (NOPASSWD). Selain itu, izin direktori /home/stack wajib diubah menggunakan chmod +x agar layanan Apache Horizon dapat membaca berkas di dalamnya.

Prosedur Pembuatan Pengguna stack pada Host (amosb)

```
sudo useradd -s /bin/bash -d /home/stack -m stack
sudo chmod +x /home/stack
echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
sudo chmod 0440 /etc/sudoers.d/stack
sudo -u stack -i
```



Gambar 3.1: Tangkapan Layar Verifikasi Akselerasi KVM (/dev/kvm) dan Pembuatan Akun Sudoers stack pada Host amosb

Pada Gambar 3.1 terlihat bahwa instruksi virtualisasi CPU terdeteksi sebanyak 4 core, berkas /dev/kvm aktif (KVM acceleration can be used), dan aturan sudoers NOPASSWD untuk pengguna stack telah berhasil dibuat.

3.3 Instalasi Paket Dependensi Dasar dan Perkakas Jaringan

Setelah beralih ke dalam sesi terminal pengguna stack (ditandai dengan prompt stack@amosb:~$), langkah selanjutnya adalah memastikan seluruh perkakas manajemen kode sumber, kompilator pustaka Python, serta utilitas diagnostik jaringan telah terpasang di sistem host. Paket-paket ini akan membantu mempercepat eksekusi skrip DevStack sekaligus menyediakan alat bantu pemecahan masalah (troubleshooting).

Instalasi Paket Prasyarat Sistem pada Host DevStack

```
sudo apt update
sudo apt install -y git vim curl wget net-tools iproute2 bridge-utils \
    python3-pip python3-dev python3-venv libffi-dev libssl-dev \
    lsof tcpdump iptables-persistent cpu-checker
```

Nama Paket Utilitas	Peran Penting dalam Operasional dan Troubleshooting DevStack
git	Mengunduh repositori DevStack serta seluruh repositori komponen OpenStack (Nova, Neutron, dll.).
net-tools & iproute2	Menyediakan perintah ip, ss, dan route untuk memeriksa tabel routing, MTU, dan port aktif.
bridge-utils	Memantau jembatan jaringan virtual Linux yang menghubungkan antarmuka tap instance.
python3-dev & pip	Mengompilasi modul pustaka klien OpenStack (python-openstackclient) dan dependensi C.
tcpdump & iptables	Menganalisis lalu lintas paket jaringan pada antarmuka br-ex dan mengatur aturan NAT/TCPMSS.
cpu-checker	Menyediakan perintah kvm-ok untuk memvalidasi kesiapan akselerasi perangkat keras KVM.

Jalankan perintah sudo kvm-ok setelah proses instalasi paket selesai. Keluaran terminal harus menampilkan pesan INFO: /dev/kvm exists dan KVM acceleration can be used, yang mengonfirmasi kesiapan hypervisor host secara penuh.

3.4 Pengaturan Parameter Kernel (sysctl) untuk Penerusan Paket Jaringan

Sebagai node tunggal (All-in-One) yang menjalankan fungsi komputasi sekaligus fungsi perutean jaringan (Network Node), kernel Linux pada host DevStack harus diizinkan untuk meneruskan paket IP (IPv4 forwarding) antar-antarmuka jaringan virtual dan fisik. Selain itu, filter Reverse Path Forwarding (rp_filter) perlu dilonggarkan agar paket asimetris dari jembatan OpenVSwitch/OVN tidak dibuang secara keliru oleh kernel.

Konfigurasi Parameter Kernel pada /etc/sysctl.conf

```
sudo tee -a /etc/sysctl.conf << 'EOF'
net.ipv4.ip_forward = 1
net.ipv4.conf.all.rp_filter = 0
net.ipv4.conf.default.rp_filter = 0
net.ipv6.conf.all.disable_ipv6 = 0
EOF

# Terapkan perubahan parameter kernel secara langsung tanpa reboot
sudo sysctl -p
```

Apabila nilai net.ipv4.ip_forward bernilai 0, maka instance OpenStack yang berada di jaringan internal (10.0.0.0/24) maupun yang telah diberikan alamat Floating IP (172.24.4.30) hanya akan mampu menghubungi alamat IP host DevStack (172.24.4.1), tetapi tidak akan pernah bisa menjangkau jaringan luar atau internet. Dengan mengaktifkan parameter di atas sejak awal, alur perutean NAT antara jembatan eksternal br-ex dan antarmuka fisik host akan berjalan tanpa hambatan.

- **Validasi Nilai Aktif:** Pastikan keluaran perintah sysctl net.ipv4.ip_forward menampilkan angka 1.
- **Persistensi Reboot:** Penulisan ke dalam berkas /etc/sysctl.conf menjamin pengaturan ini tetap bertahan meskipun mesin virtual host di-reboot.

BAB 4: PENGUNDUHAN REPOSITORI DEVSTACK DAN PENYUSUNAN LOCAL.CONF

4.1 Kloning Repositori Resmi DevStack dari OpenDev Git

Seluruh skrip otomasi DevStack dikelola secara resmi oleh OpenStack Foundation pada peladen Git OpenDev. Proses pengunduhan harus dilakukan menggunakan akun pengguna stack di dalam direktori home (/home/stack) agar kepemilikan berkas (file ownership) sesuai dengan akun yang akan mengeksekusi instalasi.

Perintah Kloning Repositori DevStack

```
# Pastikan berada di direktori home pengguna stack
cd /home/stack

# Unduh repositori DevStack dari sumber resmi OpenDev
git clone https://opendev.org/openstack/devstack

# Masuk ke dalam direktori hasil kloning
cd /home/stack/devstack

# Periksa cabang (branch) Git dan daftar berkas utama
git status
ls -la
```

Secara bawaan, perintah git clone tanpa argumen cabang akan mengunduh cabang utama (master). Apabila Anda ingin menggunakan rilis stabil tertentu yang selaras dengan versi distribusi Ubuntu host, Anda dapat menambahkan parameter -b diikuti nama cabang stabil. Setelah proses pengunduhan selesai, direktori /home/stack/devstack akan terbentuk dan berisi skrip-skrip manajemen siklus hidup OpenStack.

4.2 Bedah Struktur Direktori dan Fungsi Skrip Utilitas DevStack

Sebelum menyusun berkas konfigurasi dan menjalankan instalasi, sangat penting bagi administrator untuk memahami fungsi setiap direktori dan skrip utilitas yang berada di dalam folder /home/stack/devstack. Pemahaman ini akan sangat membantu ketika melakukan pemeliharaan layanan, pembacaan konfigurasi contoh, maupun pembersihan instalasi.

Nama Skrip / Folder	Jenis Objek	Fungsi Operasional dalam Lingkungan DevStack
stack.sh	Skrip Eksekusi	Skrip instalasi utama yang mengunduh, memasang, dan menjalankan seluruh layanan OpenStack.
unstack.sh	Skrip Eksekusi	Menghentikan seluruh layanan OpenStack yang sedang berjalan tanpa menghapus data konfigurasi.
clean.sh	Skrip Eksekusi	Membersihkan lingkungan instalasi, menghapus konfigurasi sementara, dan mereset status sistem.
openrc	Skrip Lingkungan	Mengatur variabel lingkungan CLI (OS_AUTH_URL, OS_USERNAME, dll.) untuk otorisasi perintah openstack.
samples/	Direktori	Berisi contoh berkas konfigurasi local.conf dan local.sh siap pakai sebagai referensi awal.
lib/	Direktori	Berisi modul skrip fungsi spesifik untuk setiap layanan (lib/nova, lib/neutron, lib/keystone, lib/databases).
tools/	Direktori	Berisi perkakas bantu diagnostik, pembuatan akun sistem, dan manajemen citra disk.
files/	Direktori	Tempat penyimpanan berkas citra OS (image) yang diunduh selama proses instalasi Glance.

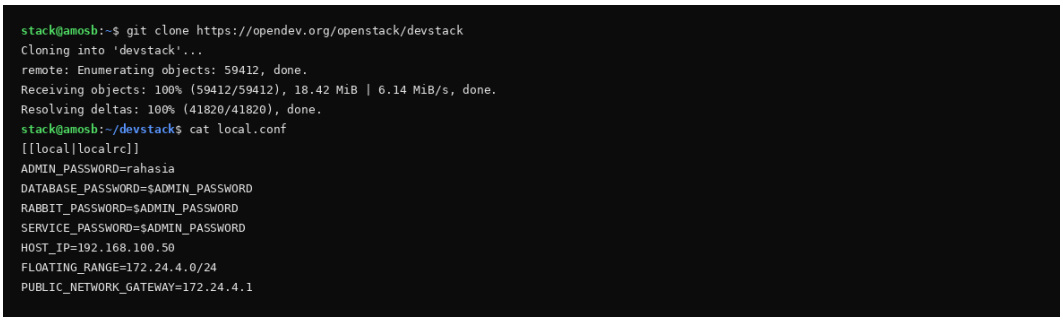
Di antara seluruh berkas di atas, berkas openrc adalah berkas yang paling sering dipanggil dalam kegiatan operasional sehari-hari. Setiap kali Anda membuka sesi terminal baru pada host stack@amosb dan ingin menjalankan perintah manajemen seperti openstack server list atau openstack security group rule create, Anda wajib menjalankan perintah source openrc admin admin terlebih dahulu.

4.3 Penyusunan Berkas Konfigurasi local.conf Langkah Demi Langkah

Seluruh perilaku instalasi DevStack dikendalikan oleh sebuah berkas bernama local.conf yang ditempatkan di akar direktori /home/stack/devstack/. Berkas ini wajib dimulai dengan baris header [[local|localrc]] pada baris paling atas. Di bawah header tersebut, kita menetapkan kata sandi layanan, alamat IP host, serta parameter rentang jaringan virtual Neutron.

Pembuatan Berkas /home/stack/devstack/local.conf

```
cat << 'EOF' > /home/stack/devstack/local.conf
[[local|localrc]]
ADMIN_PASSWORD=rahasia
DATABASE_PASSWORD=$ADMIN_PASSWORD
RABBIT_PASSWORD=$ADMIN_PASSWORD
SERVICE_PASSWORD=$ADMIN_PASSWORD
HOST_IP=192.168.100.50
FLOATING_RANGE=172.24.4.0/24
PUBLIC_NETWORK_GATEWAY=172.24.4.1
EOF
```



Gambar 4.1: Tangkapan Layar Kloning Repositori Git DevStack dan Pemeriksaan Isi Berkas local.conf

Pastikan nilai FLOATING_RANGE=172.24.4.0/24 dan PUBLIC_NETWORK_GATEWAY=172.24.4.1 tertulis dengan benar seperti pada Gambar 4.1 karena rentang inilah yang digunakan oleh jembatan br-ex (172.24.4.1) dan Floating IP server-kedua (172.24.4.30).

4.4 Penjelasan Detail Setiap Parameter dalam Berkas local.conf

Agar tidak terjadi kesalahan interpretasi saat mengelola jaringan dan autentikasi OpenStack, tabel berikut menjelaskan arti teknis dari setiap variabel konfigurasi yang telah dituliskan ke dalam berkas local.conf.

Variabel local.conf	Contoh Nilai	Penjelasan Teknis dan Dampak pada Sistem OpenStack
ADMIN_PASSWORD	rahasia	Kata sandi utama untuk login akun admin pada dashboard web Horizon dan CLI OpenStack.
DATABASE_PASSWORD	$ADMIN_PASSWORD	Kata sandi akun root pada MySQL Server host yang menyimpan database nova, neutron, keystone.
RABBIT_PASSWORD	$ADMIN_PASSWORD	Kata sandi autentikasi untuk message broker RabbitMQ yang menghubungkan komponen antar-layanan.
SERVICE_PASSWORD	$ADMIN_PASSWORD	Kata sandi internal yang digunakan oleh agen layanan (seperti nova-compute) untuk melapor ke Keystone.
HOST_IP	192.168.100.50	Alamat IPv4 antarmuka utama host tempat seluruh layanan API OpenStack mendengarkan permintaan.
FLOATING_RANGE	172.24.4.0/24	Blok subnet jaringan eksternal (public) yang menyediakan alamat Floating IP bagi instance.
PUBLIC_NETWORK_GATEWAY	172.24.4.1	Alamat IP yang dipasang pada jembatan br-ex di host DevStack sebagai gateway keluar instance.
IPV4_ADDRS_SAFE_TO_USE	10.0.0.0/24	Blok subnet jaringan privat (internal) yang diberikan secara otomatis ke antarmuka ens3 instance.
LOGFILE	$DEST/logs/stack.sh.log	Lokasi penyimpanan catatan log lengkap proses instalasi di /opt/stack/logs/stack.sh.log.

Dengan mendefinisikan parameter di atas secara eksplisit, skrip stack.sh tidak akan meminta input interaktif di tengah proses instalasi sehingga proses pembangunan cloud dapat berjalan secara otomatis dari awal hingga selesai.

BAB 5: EKSEKUSI INSTALASI DEVSTACK (STACK.SH) DAN VERIFIKASI LAYANAN

5.1 Prosedur Eksekusi Skrip ./stack.sh dan Tahapan Otomatisasi

Setelah berkas local.conf tersimpan dengan benar di dalam direktori /home/stack/devstack, proses instalasi OpenStack dapat segera dimulai. Proses ini membutuhkan waktu sekitar 20 hingga 45 menit bergantung pada kecepatan koneksi internet dan performa I/O penyimpanan mesin virtual. Jalankan skrip instalasi menggunakan perintah berikut.

Perintah Eksekusi Instalasi DevStack

```
cd /home/stack/devstack
./stack.sh
```

Selama skrip ./stack.sh berjalan, sistem akan mengeksekusi serangkaian tahapan sistematis secara berurutan:

- **Tahap 1 (Validasi Sistem & Repositori APT):** Memeriksa distribusi OS Ubuntu, menambahkan repositori paket cloud, dan memasang paket sistem seperti MySQL Server, RabbitMQ, Apache2, OpenVSwitch/OVN, serta Libvirt/QEMU.
- **Tahap 2 (Kloning Kode Sumber Komponen):** Mengunduh kode sumber Keystone, Glance, Nova, Placement, Cinder, Neutron, dan Horizon ke dalam direktori /opt/stack/.
- **Tahap 3 (Inisialisasi Basis Data MySQL):** Membuat skema basis data untuk masing-masing layanan di dalam MySQL host dan menjalankan migrasi tabel (db_sync).
- **Tahap 4 (Konfigurasi Keystone & Katalog Endpoint):** Mendaftarkan proyek admin, demo, service, serta membuat endpoint URL untuk semua API.
- **Tahap 5 (Konfigurasi Jaringan Neutron & Image Glance):** Membangun jembatan OVS br-ex (172.24.4.1), membuat jaringan public dan private, serta mengunduh citra uji Cirros.

5.2 Pemantauan Log Instalasi dan Bukti Keberhasilan ./stack.sh

Ketika ./stack.sh sedang berjalan di terminal utama, Anda dapat memantau log instalasi secara langsung melalui berkas /opt/stack/logs/stack.sh.log. Apabila proses instalasi berhasil diselesaikan tanpa kendala, bagian akhir keluaran terminal akan menampilkan ringkasan DevStack Component Timing beserta alamat URL Dashboard Horizon dan Keystone Identity seperti yang ditunjukkan pada Gambar 5.1.

Perintah Pemantauan Log dan Pembersihan Jika Terjadi Timeout

```
tail -f /opt/stack/logs/stack.sh.log
# Jika perlu mengulang instalasi akibat gangguan koneksi:
./unstack.sh && ./clean.sh && ./stack.sh
```



Gambar 5.1: Tangkapan Layar Penyelesaian Instalasi ./stack.sh dan Aktivasi Kredensial openrc pada Host amosb

Tampilnya pesan stack.sh completed pada Gambar 5.1 menandakan bahwa seluruh layanan inti OpenStack telah aktif dan siap dioperasikan melalui Horizon Web UI maupun melalui OpenStack CLI.

5.3 Verifikasi Layanan Systemd DevStack pada Host (amosb)

Pada arsitektur DevStack modern, seluruh proses latar belakang (daemon) OpenStack dikelola sebagai unit layanan systemd dengan awalan nama devstack@. Selain itu, layanan pendukung infrastruktur seperti MySQL, RabbitMQ, Apache2, dan Libvirtd juga berjalan sebagai unit systemd standar. Setelah ./stack.sh selesai, lakukan verifikasi menyeluruh menggunakan perintah systemctl.

Pemeriksaan Status Unit Layanan DevStack dan Infrastruktur Host

```
# 1. Tampilkan seluruh unit layanan devstack yang sedang aktif
systemctl list-units 'devstack@*' --no-pager

# 2. Periksa status layanan basis data utama MySQL DevStack
sudo systemctl status mysql --no-pager

# 3. Periksa status message broker RabbitMQ dan Web Server Apache
sudo systemctl status rabbitmq-server apache2 --no-pager

# 4. Periksa status hypervisor Libvirt/KVM
sudo systemctl status libvirtd --no-pager
```

Nama Unit Systemd	Komponen Terkait	Indikator Status Normal
devstack@n-cpu.service	Nova Compute	active (running) - Mengelola eksekusi VM KVM
devstack@n-api.service	Nova API	active (running) - Melayani permintaan komputasi
devstack@g-api.service	Glance API	active (running) - Melayani manajemen citra OS
devstack@q-svc.service	Neutron Server	active (running) - Mengelola jaringan virtual & Security Group
mysql.service	MySQL Server 8.0	active (running) - Menyimpan seluruh metadata OpenStack

5.4 Pengujian Akses Antarmuka Grafis Dashboard Horizon Web UI

OpenStack Horizon menyediakan portal administrasi berbasis web yang memudahkan visualisasi topologi jaringan, pemantauan log konsol instance, serta manajemen volume dan Security Group. Untuk mengakses Horizon, buka peramban web (browser) pada sistem operasi Windows utama Anda, lalu masukkan alamat URL http://<IP_HOST_DEVSTACK>/dashboard.

- **1. Halaman Login Horizon:** Masukkan Domain (jika diminta) dengan nilai Default, User Name dengan nilai admin, dan Password sesuai nilai ADMIN_PASSWORD di local.conf, lalu klik Sign In.
- **2. Pemilih Proyek (Project Selector):** Di pojok kiri atas halaman, pastikan proyek yang sedang aktif adalah admin (untuk melihat seluruh sumber daya sistem) atau demo.
- **3. Menu Admin -> System -> System Information:** Buka menu ini untuk memverifikasi bahwa seluruh layanan pada tab Services, Compute Services, Block Storage Services, dan Network Agents berstatus Enabled dan Up.
- **4. Menu Project -> Network -> Network Topology:** Periksa visualisasi grafis jaringan yang menampilkan keterhubungan antara jaringan eksternal public (172.24.4.0/24), router virtual router1, dan jaringan internal private.

Meskipun antarmuka Horizon sangat nyaman untuk pemantauan visual, seorang administrator sistem dan rekayasawan cloud harus menguasai penggunaan antarmuka baris perintah (OpenStack CLI) karena CLI memberikan kontrol yang jauh lebih presisi, mendukung otomatisasi skrip, dan menampilkan pesan diagnostik yang lengkap.

5.5 Penggunaan Berkas openrc dan Verifikasi OpenStack CLI

Agar perintah openstack dapat dijalankan dari terminal host stack@amosb:~/devstack$, kita harus memuat variabel lingkungan autentikasi Keystone terlebih dahulu menggunakan berkas skrip openrc. Skrip ini menerima dua argumen posisi, yaitu nama pengguna (username) dan nama proyek (project/tenant).

Aktivasi Kredensial Admin dan Verifikasi Endpoint CLI

```
cd /home/stack/devstack

# Muat kredensial sebagai pengguna 'admin' pada proyek 'admin'
source openrc admin admin

# Verifikasi variabel lingkungan yang telah dimuat
env | grep OS_

# Uji komunikasi ke Keystone, Nova, Neutron, dan Glance
openstack endpoint list
openstack compute service list
openstack network agent list
```

Ketika perintah source openrc admin admin dijalankan, sistem mengatur variabel seperti OS_AUTH_URL=http://<IP_HOST>/identity, OS_USERNAME=admin, OS_PROJECT_NAME=admin, dan OS_PASSWORD. Apabila keluaran perintah openstack compute service list menampilkan tabel layanan nova-conductor, nova-scheduler, dan nova-compute pada host amosb dengan status enabled dan state up, maka seluruh fondasi OpenStack telah siap digunakan untuk memuat citra Ubuntu dan meluncurkan instance.

BAB 6: MANAJEMEN CITRA SISTEM OPERASI (GLANCE) DAN FLAVOR (NOVA)

6.1 Konsep Citra Sistem Operasi (Cloud Image) pada OpenStack Glance

Secara bawaan, instalasi DevStack hanya menyertakan satu citra sistem operasi minimalis bernama Cirros (sekitar 16 MB). Cirros sangat berguna untuk menguji fungsi booting cepat dan konektivitas ping, tetapi tidak memiliki manajer paket APT maupun pustaka sistem yang dibutuhkan untuk memasang perangkat lunak produksi seperti peladen basis data MariaDB. Oleh sebab itu, untuk membangun server-pertama dan server-kedua, kita perlu mengunggah citra resmi Ubuntu Cloud Image (Ubuntu 22.04 LTS Jammy Jellyfish) ke dalam layanan OpenStack Glance.

Karakteristik Citra	Citra Bawaan Cirros	Ubuntu 22.04 LTS Cloud Image (Jammy)
Ukuran Berkas Disk	~16 MB - 20 MB	~650 MB (Terkompresi QCOW2)
Kegunaan Utama	Uji fungsi boot cepat dan ping jaringan	Server aplikasi, web server, dan database server (MariaDB)
Manajer Paket	Tidak tersedia (BusyBox statis)	Tersedia penuh (APT / dpkg dengan repositori Ubuntu)
Inisialisasi Cloud	Skrip init sederhana	Dukungan penuh paket cloud-init (injeksi SSH key & hostname)
Akun Pengguna Default	cirros (sandi: gocubsgo)	ubuntu (autentikasi kunci SSH / cloud-init)

Citra Ubuntu Cloud Image dirancang khusus untuk lingkungan cloud IaaS. Citra ini menggunakan format disk QEMU Copy-On-Write versi 2 (QCOW2) yang mendukung alokasi ruang tipis (thin provisioning) dan telah dilengkapi layanan cloud-init yang secara otomatis mengatur hostname instance (seperti server-kedua) serta menyuntikkan kunci publik SSH saat pertama kali dinyalakan.

6.2 Pengunduhan dan Pendaftaran Citra Ubuntu 22.04 LTS ke Glance

Proses pendaftaran citra Ubuntu 22.04 LTS dilakukan langsung dari terminal host DevStack (stack@amosb:~/devstack$). Kita mengunduh berkas jammy-server-cloudimg-amd64.img dari peladen rilis resmi Canonical menggunakan wget, kemudian mengunggahnya ke layanan Glance menggunakan perintah openstack image create.

Prosedur Unduh dan Unggah Citra Ubuntu 22.04 LTS pada Host DevStack

```
cd /home/stack/devstack && source openrc admin admin
wget -c https://cloud-images.ubuntu.com/jammy/current/jammy-server-cloudimg-
amd64.img \
     -O ~/jammy-server-cloudimg-amd64.img
openstack image create "Ubuntu-22.04" --file ~/jammy-server-cloudimg-amd64.img \
  --disk-format qcow2 --container-format bare --public
openstack image list
```



Gambar 6.1: Tangkapan Layar Pendaftaran Citra Ubuntu-22.04 dan Daftar Image Aktif di Glance

Sebagaimana terlihat pada Gambar 6.1, citra Ubuntu-22.04 telah terdaftar di dalam katalog Glance dengan status active berdampingan dengan citra bawaan cirros.

6.3 Inspeksi Properti Citra dan Penyimpanan Backend Glance

Setelah citra Ubuntu-22.04 berstatus active, kita dapat memeriksa rincian atribut teknis citra tersebut seperti ukuran byte, nilai hash checksum MD5/SHA512, serta lokasi penyimpanan fisik berkas di dalam direktori /opt/stack/data/glance/images/. Pemeriksaan ini memastikan berkas citra tidak mengalami kerusakan (corrupt) selama proses pengunduhan.

Perintah Inspeksi Detail Citra pada Glance

```
# Tampilkan metadata lengkap citra Ubuntu-22.04
openstack image show "Ubuntu-22.04"

# Periksa informasi struktur virtual disk menggunakan qemu-img
qemu-img info /home/stack/jammy-server-cloudimg-amd64.img
```

Contoh Keluaran Informasi Struktur Virtual Disk (qemu-img info)

```
image: /home/stack/jammy-server-cloudimg-amd64.img
file format: qcow2
virtual size: 2.2 GiB (2361393152 bytes)
disk size: 648 MiB
cluster_size: 65536
Format specific information:
    compat: 1.1
    compression type: zlib
```

Terlihat bahwa ukuran fisik berkas adalah sekitar 648 MiB, sedangkan ukuran virtual awalnya adalah 2.2 GiB. Ketika citra ini diluncurkan menggunakan sebuah Flavor Nova yang memiliki kapasitas disk lebih besar (misalnya 10 GB atau 20 GB), paket cloud-init/growpart di dalam Ubuntu akan secara otomatis memperluas partisi root (/) hingga memenuhi seluruh kapasitas disk Flavor yang diberikan.

6.4 Manajemen dan Pembuatan Flavor Komputasi pada OpenStack Nova

Di dalam OpenStack Nova, spesifikasi perangkat keras virtual yang dialokasikan untuk sebuah instance disebut sebagai Flavor. Sebuah Flavor mendefinisikan tiga parameter utama: jumlah prosesor virtual (vCPU), kapasitas memori utama (RAM dalam satuan MB), dan kapasitas penyimpanan root disk (dalam satuan GB). Untuk menjalankan Ubuntu 22.04 beserta layanan basis data MariaDB secara responsif pada server-kedua, kita membutuhkan Flavor dengan RAM minimal 1024 MB hingga 2048 MB dan disk minimal 10 GB.

Pemeriksaan dan Pembuatan Flavor Komputasi untuk Server Ubuntu

```
# 1. Tampilkan daftar flavor bawaan yang ada di sistem
openstack flavor list

# 2. Buat flavor khusus jika diperlukan (atau gunakan m1.small / m1.medium)
openstack flavor create --id 10 --ram 2048 --disk 15 --vcpus 2 m1.dbserver

# 3. Verifikasi detail spesifikasi flavor
openstack flavor show m1.small
```

Nama Flavor	vCPUs	RAM (MB)	Root Disk (GB)	Rekomendasi Penggunaan Instance
m1.nano / m1.tiny	1	128 - 512	1	Khusus pengujian citra ringan Cirros
m1.small	1	2048	20	Ideal untuk server-pertama dan server-kedua (Ubuntu + MariaDB)
m1.medium	2	4096	40	Beban kerja menengah atau server kompilasi aplikasi
m1.dbserver	2	2048	15	Flavor kustom teroptimasi untuk laboratorium multi-instance

Dengan tersedianya citra Ubuntu-22.04 dan Flavor m1.small / m1.dbserver, kebutuhan komputasi dan sistem operasi untuk instance telah terpenuhi. Tahap selanjutnya adalah menyiapkan infrastruktur jaringan virtual dan keamanan di Neutron.

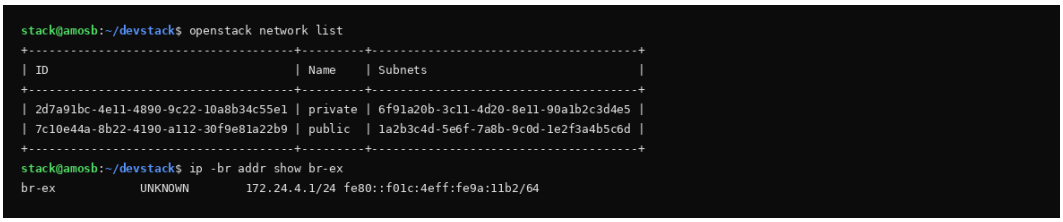
BAB 7: KONFIGURASI JARINGAN VIRTUAL NEUTRON, ROUTER, DAN KEAMANAN

7.1 Arsitektur Jaringan Eksternal (Public) dan Internal (Private)

OpenStack Neutron mengelola komunikasi jaringan virtual menggunakan pemisahan tegas antara dua jenis jaringan utama: Jaringan Internal (private) dan Jaringan Eksternal (public). Setiap instance yang diluncurkan memiliki antarmuka jaringan virtual (ens3) yang terhubung ke jaringan private (10.0.0.0/24) dan dipetakan ke jaringan public (172.24.4.0/24) melalui router virtual Neutron.

Pemeriksaan Daftar Jaringan Neutron dan Antarmuka Bridge br-ex

```
source openrc admin admin
openstack network list
ip -br addr show br-ex
```



Gambar 7.1: Tangkapan Layar Daftar Jaringan Neutron dan Alamat IP Gateway 172.24.4.1 pada Jembatan br-ex

Pada Gambar 7.1 terlihat bahwa jembatan br-ex di host DevStack memegang alamat IP 172.24.4.1/24. Alamat ini menjadi titik temu langsung antara host DevStack dengan alamat Floating IP instance server-kedua (172.24.4.30).

7.2 Konfigurasi Jaringan Privat, Subnet, dan DNS Nameserver Neutron

Agar instance yang berada di dalam jaringan privat (private-subnet) dapat menerjemahkan nama domain internet seperti mirror.biznetgio.com atau archive.ubuntu.com menjadi alamat IP, layanan DHCP milik Neutron harus menyertakan alamat peladen DNS (Nameserver) ketika memberikan alamat IP kepada antarmuka ens3 instance. Berikut adalah langkah pemeriksaan dan pengaturan DNS nameserver pada subnet privat.

Konfigurasi DNS Nameserver pada Subnet Privat OpenStack

```
# 1. Lihat detail konfigurasi subnet privat bawaan (ipv4-private-subnet / private-subnet)
openstack subnet show private-subnet

# 2. Tambahkan DNS Nameserver publik (Google DNS & Cloudflare) ke subnet privat
openstack subnet set --dns-nameserver 8.8.8.8 --dns-nameserver 1.1.1.1 private-subnet

# 3. Jika ingin membuat jaringan dan subnet privat baru secara manual:
# openstack network create net-internal
# openstack subnet create --network net-internal --subnet-range 10.0.0.0/24 \
#   --gateway 10.0.0.1 --dns-nameserver 8.8.8.8 sub-internal
```

Dengan menyetel parameter --dns-nameserver pada private-subnet, daemon systemd-resolved di dalam instance Ubuntu 22.04 akan secara otomatis menerima informasi peladen DNS dari paket tawaran DHCP Neutron saat proses booting berlangsung.

- **Alokasi IP Otomatis:** Agen DHCP Neutron mendengarkan permintaan DHCP Discover di jaringan internal dan memberikan sewa IP dari rentang alokasi 10.0.0.2 hingga 10.0.0.254.
- **Rute Default Instance:** Gateway default di dalam instance akan diarahkan ke alamat IP port internal router virtual Neutron (10.0.0.1).

7.3 Verifikasi Virtual Router Neutron dan Aturan NAT Masquerade Host

Router virtual Neutron (router1) memiliki dua kaki antarmuka: satu kaki terhubung sebagai External Gateway pada jaringan public (172.24.4.0/24), dan satu kaki lagi terhubung sebagai Internal Interface pada jaringan private (10.0.0.1/24). Namun, agar paket dari subnet 172.24.4.0/24 yang tiba di jembatan br-ex milik host DevStack (172.24.4.1) dapat diteruskan keluar melalui kartu jaringan fisik host (ens33) menuju internet, tabel iptables NAT pada host DevStack harus memiliki aturan MASQUERADE.

Verifikasi Router Neutron dan Aturan Iptables NAT pada Host

```
# 1. Periksa status router virtual di OpenStack
openstack router list
openstack router show router1

# 2. Pastikan aturan NAT Masquerade aktif pada antarmuka keluar host (misal
ens33)
sudo iptables -t nat -C POSTROUTING -s 172.24.4.0/24 ! -d 172.24.4.0/24 -j
MASQUERADE 2>/dev/null || \
sudo iptables -t nat -A POSTROUTING -s 172.24.4.0/24 ! -d 172.24.4.0/24 -j
MASQUERADE

# 3. Verifikasi tabel POSTROUTING NAT di host
sudo iptables -t nat -L POSTROUTING -n -v
```

Aturan iptables MASQUERADE di atas memastikan setiap paket yang berasal dari jaringan Floating IP OpenStack (172.24.4.0/24) yang menuju ke alamat luar diterjemahkan alamat sumbernya menjadi alamat IP host DevStack sebelum dikirim ke gateway VMware.

7.4 Pembuatan dan Pendaftaran Pasangan Kunci SSH (Keypair)

Citra resmi Ubuntu Cloud Image menonaktifkan autentikasi kata sandi teks biasa (PasswordAuthentication no) secara bawaan demi alasan keamanan. Metode masuk yang diizinkan untuk pengguna ubuntu adalah autentikasi kunci publik kriptografi SSH. Oleh karena itu, kita membuat pasangan kunci SSH pada akun stack di host DevStack dan mendaftarkan kunci publiknya ke dalam OpenStack Nova.

Pembuatan Kunci SSH di Host dan Pendaftaran Keypair OpenStack

```
# 1. Buat pasangan kunci RSA/Ed25519 di host DevStack jika belum ada
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa -N ""

# 2. Daftarkan kunci publik ke dalam OpenStack Nova
openstack keypair create --public-key ~/.ssh/id_rsa.pub kunci-ssh-amos

# 3. Verifikasi daftar keypair yang terdaftar
openstack keypair list
```

Ketika sebuah instance diluncurkan dengan menyertakan parameter --key-name kunci-ssh-amos, layanan metadata Nova akan menyerahkan isi berkas ~/.ssh/id_rsa.pub kepada layanan cloud-init di dalam instance. Selanjutnya, cloud-init menuliskan kunci publik tersebut ke dalam berkas /home/ubuntu/.ssh/authorized_keys di dalam server-kedua, sehingga kita dapat langsung masuk dari host DevStack menggunakan perintah ssh ubuntu@172.24.4.30 tanpa diminta kata sandi.

7.5 Konfigurasi Aturan Dasar Security Group (ICMP Ping & SSH Port 22)

Secara bawaan, grup keamanan (Security Group) di OpenStack menerapkan kebijakan tolak semua trafik masuk (Deny All Ingress) dan izinkan semua trafik keluar (Allow All Egress). Artinya, meskipun sebuah instance sudah menyala dan memiliki Floating IP, instance tersebut tidak akan merespons uji ping (ICMP) maupun koneksi SSH (TCP Port 22) sebelum aturan masuk ditambahkan ke dalam Security Group.

Pembukaan Akses ICMP (Ping) dan TCP Port 22 (SSH) pada Security Group

```
source openrc admin admin

# 1. Ambil ID Security Group 'default' milik proyek aktif saat ini
SG_DEFAULT=$(openstack security group list --project admin -f value -c ID -c Name
| grep default | awk '{print $1}')
echo "ID Security Group Default: $SG_DEFAULT"

# 2. Tambahkan aturan izinkan protokol ICMP (Ping) dari semua alamat IP
openstack security group rule create --proto icmp --remote-ip 0.0.0.0/0
$SG_DEFAULT

# 3. Tambahkan aturan izinkan protokol TCP Port 22 (SSH) dari semua alamat IP
openstack security group rule create --proto tcp --dst-port 22 --remote-ip
0.0.0.0/0 $SG_DEFAULT

# 4. Tampilkan aturan Security Group yang sudah terpasang
openstack security group show $SG_DEFAULT
```

Perhatikan penggunaan variabel $SG_DEFAULT di atas. Dalam instalasi OpenStack, setiap proyek (seperti proyek admin, demo, dan service) masing-masing memiliki Security Group bernama default. Menyimpan UUID spesifik ke dalam variabel menghindarkan kita dari galat More than one SecurityGroup exists with the name 'default'.

BAB 8: PEMBUATAN DAN PELUNCURAN INSTANCE (SERVER-PERTAMA & SERVER-KEDUA)

8.1 Tahapan Peluncuran Instance Melalui Dashboard Horizon dan CLI

Dengan tersedianya citra Ubuntu-22.04, Flavor m1.small, jaringan privat, pasangan kunci SSH, dan aturan Security Group dasar, kita siap meluncurkan instance mesin virtual. Pembuatan instance dapat dilakukan melalui dua cara yang setara: melalui wizard grafis Launch Instance pada Dashboard Horizon, atau melalui satu baris perintah openstack server create pada terminal host DevStack.

Tab Wizard Horizon	Parameter Pilihan	Padanan Argumen pada OpenStack CLI
1. Details	Instance Name: server-kedua	Argumen posisi nama di akhir perintah: server-kedua
2. Source	Boot Source: Image -> Ubuntu-22.04	--image "Ubuntu-22.04"
3. Flavor	Allocated Flavor: m1.small	--flavor m1.small
4. Networks	Allocated Network: private	--network private
5. Security Groups	Selected Group: default	--security-group $SG_DEFAULT
6. Key Pair	Allocated Key Pair: kunci-ssh-amos	--key-name kunci-ssh-amos

Saat perintah peluncuran dikirimkan, Nova API mencatat permintaan ke basis data MySQL host, Nova Scheduler memilih node komputasi amosb, Glance mengirimkan berkas citra Ubuntu-22.04 ke cache lokal Nova, Neutron membuat port virtual beserta alokasi IP privat, dan akhirnya Nova Compute memerintahkan hypervisor Libvirt/KVM untuk menghidupkan mesin virtual.

8.2 Eksekusi Peluncuran Instance server-pertama dan server-kedua

Di dalam terminal host DevStack (stack@amosb:~/devstack$), jalankan perintah berikut untuk meluncurkan instance pertama (server-pertama) dan instance kedua (server-kedua) yang akan difungsikan sebagai peladen basis data MariaDB.

Perintah Peluncuran Instance server-pertama dan server-kedua

```
cd /home/stack/devstack
source openrc admin admin

# Dapatkan ID jaringan privat dan Security Group default proyek admin
NET_PRIV=$(openstack network list --name private -f value -c ID)
SG_DEFAULT=$(openstack security group list --project admin -f value -c ID -c Name
| grep default | awk '{print $1}')

# 1. Luncurkan instance server-pertama
openstack server create \
  --image "Ubuntu-22.04" \
  --flavor m1.small \
  --network $NET_PRIV \
  --security-group $SG_DEFAULT \
  --key-name kunci-ssh-amos \
  server-pertama

# 2. Luncurkan instance server-kedua
openstack server create \
  --image "Ubuntu-22.04" \
  --flavor m1.small \
  --network $NET_PRIV \
  --security-group $SG_DEFAULT \
  --key-name kunci-ssh-amos \
  server-kedua

# 3. Pantau status pembangunan instance (BUILD -> ACTIVE)
openstack server list
```

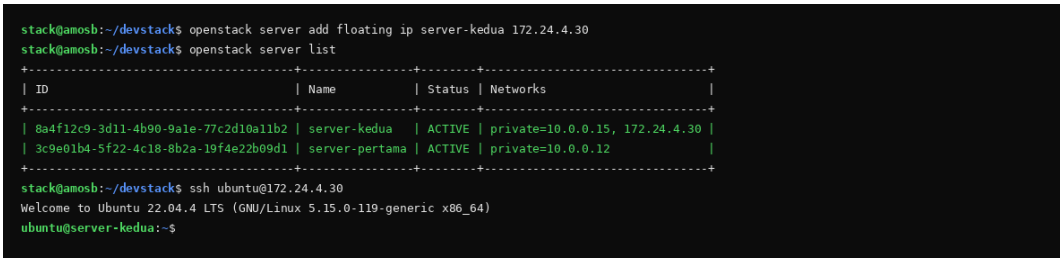
Tunggu sekitar 15 hingga 30 detik lalu jalankan kembali perintah openstack server list hingga kolom Status berubah dari BUILD menjadi ACTIVE dan kolom Power State menunjukkan Running.

8.3 Alokasi dan Asosiasi Alamat Floating IP (172.24.4.30) ke server-kedua

Setelah instance server-kedua berstatus ACTIVE, instance tersebut baru memiliki alamat IP privat pada antarmuka ens3 (dari subnet 10.0.0.0/24). Agar kita dapat melakukan koneksi SSH langsung dari host DevStack (172.24.4.1) serta menguji koneksi klien basis data dari luar, kita mengalokasikan alamat Floating IP 172.24.4.30 dari jaringan public dan memasangkannya ke server-kedua.

Alokasi dan Pemasangan Floating IP 172.24.4.30 pada server-kedua

```
openstack floating ip create public --floating-ip-address 172.24.4.30
openstack server add floating ip server-kedua 172.24.4.30
openstack server list
```



Gambar 8.1: Tangkapan Layar Asosiasi Floating IP 172.24.4.30 pada server-kedua dan Login SSH Pertama

Terlihat pada Gambar 8.1 bahwa server-kedua telah berstatus ACTIVE dengan jaringan private=10.0.0.15, 172.24.4.30 dan sesi SSH berhasil masuk ke prompt ubuntu@server-kedua:~$.

8.4 Pemeriksaan Log Konsol Booting dan Inisialisasi Cloud-Init

Saat sebuah instance Ubuntu 22.04 pertama kali berstatus ACTIVE, proses di dalam sistem operasi tamu (guest OS) masih menjalankan skrip inisialisasi awal cloud-init selama sekitar 20 hingga 40 detik untuk membuat kunci host SSH dan menyalin kunci publik pengguna. Kita dapat memastikan proses booting di dalam server-kedua telah selesai sepenuhnya dengan membaca log konsol langsung dari host DevStack.

Pemeriksaan Log Konsol Booting Instance server-kedua

```
# Tampilkan 30 baris terakhir log konsol virtual server-kedua
openstack console log show server-kedua | tail -n 30
```

Potongan Log Konsol yang Menandakan Instance Siap Diakses

```
[   18.412091] cloud-init[742]: Cloud-init v. 24.1.3-0ubuntu1~22.04.1 running
'modules:final'
ci-info: +++++++Authorized keys from /home/ubuntu/.ssh/authorized_keys for user
ubuntu+++++++
ci-info: +---------+-------------------------------------------------+---------+-
------------------+
ci-info: | Keytype |               Fingerprint (md5)                 | Options |
Comment      |
ci-info: +---------+-------------------------------------------------+---------+-
------------------+
ci-info: | ssh-rsa | 4a:8b:91:c2:d3:e4:f5:01:23:45:67:89:ab:cd:ef:01 |    -    |
stack@amosb     |
ci-info: +---------+-------------------------------------------------+---------+-
------------------+
[   19.104512] cloud-init[742]: Cloud-init v. 24.1.3 finished at Sun, 25 Oct 2026
13:15:10 +0000.
Ubuntu 22.04.4 LTS server-kedua ttyS0
server-kedua login:
```

Munculnya tabel Authorized keys dan baris server-kedua login: pada log konsol menandakan bahwa layanan SSH di dalam server-kedua telah aktif dan siap menerima koneksi masuk.

8.5 Pengujian Konektivitas Ping dan Login SSH ke server-kedua (172.24.4.30)

Dari terminal host DevStack (stack@amosb:~/devstack$), lakukan pengujian paket ICMP ping menuju alamat Floating IP 172.24.4.30, kemudian lanjutkan dengan membuka sesi remote shell SSH menggunakan akun pengguna bawaan citra Ubuntu yaitu ubuntu.

Uji Ping dan Koneksi SSH dari Host DevStack ke server-kedua

```
# 1. Uji konektivitas jaringan lapisan IP menuju Floating IP server-kedua
ping -c 4 172.24.4.30

# 2. Masuk ke dalam instance server-kedua melalui SSH
ssh ubuntu@172.24.4.30
```

Tampilan Terminal Saat Berhasil Masuk ke Instance server-kedua

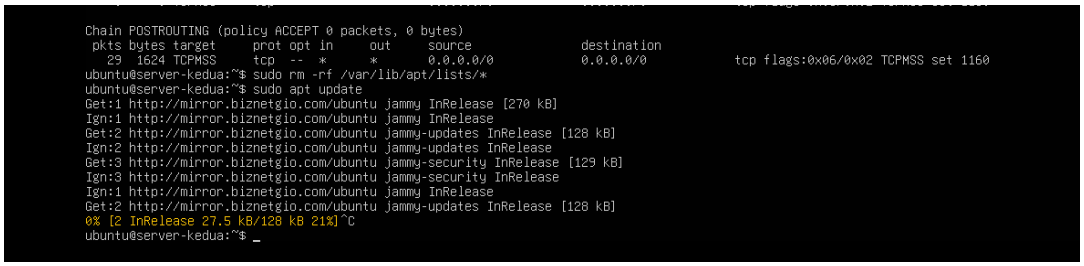
```
The authenticity of host '172.24.4.30 (172.24.4.30)' can't be established.
ED25519 key fingerprint is SHA256:xK9mP2vL8nQ4rT6wY1zB3cV5bN7mJ0kL2pQ4rS6tU8v.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.24.4.30' (ED25519) to the list of known hosts.
Welcome to Ubuntu 22.04.4 LTS (GNU/Linux 5.15.0-119-generic x86_64)
...
ubuntu@server-kedua:~$
```

Perubahan prompt terminal dari stack@amosb:~/devstack$ menjadi ubuntu@server-kedua:~$ mengonfirmasi bahwa kita telah berada di dalam sistem operasi instance server-kedua. Selanjutnya, kita akan melakukan pembaruan repositori sistem sebelum memasang basis data MariaDB.

BAB 9: DIAGNOSIS DAN PENYELESAIAN KENDALA PATH MTU BLACK HOLE

9.1 Gejala Kemacetan Unduhan APT (Stuck di 8% - 21%) pada server-kedua

Setelah berhasil masuk ke dalam terminal ubuntu@server-kedua:~$, langkah standar administrasi sistem sebelum memasang paket perangkat lunak baru adalah memperbarui indeks repositori APT menggunakan perintah sudo apt update. Untuk mempercepat pengunduhan di wilayah Indonesia, repositori diarahkan ke peladen mirror lokal (http://mirror.biznetgio.com/ubuntu). Namun, saat perintah sudo apt update dijalankan, proses pengunduhan selalu macet dan berhenti merespons ketika baru mencapai ukuran sekitar 10.7 kB hingga 27.5 kB (21%).



Gambar 9.1: Tangkapan Layar Asli Kemacetan Unduhan sudo apt update pada 27.5 kB (21%) di server-kedua

Sebagaimana terekam secara nyata pada Gambar 9.1, perintah sudo rm -rf /var/lib/apt/lists/* diikuti sudo apt update berhasil membuka koneksi awal ke mirror.biznetgio.com, namun terhenti total pada baris 0% [2 InRelease 27.5 kB/128 kB 21%] hingga harus dihentikan paksa dengan Ctrl+C (^C). Gejala ini adalah indikasi klasik dari Path MTU Black Hole pada jaringan virtualisasi bertingkat.

9.2 Analisis Teknis Enkapsulasi Overlay Neutron dan Path MTU Black Hole

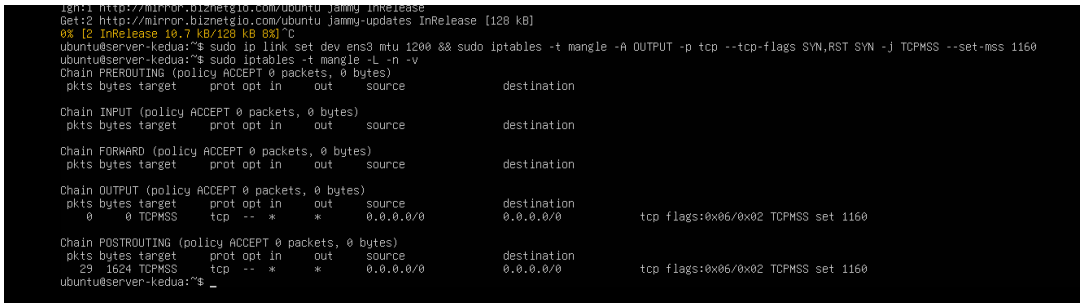
Mengapa unduhan selalu berhenti tepat di kisaran 10 kB hingga 27 kB? Dalam arsitektur OpenStack DevStack yang berjalan di atas mesin virtual VMware (nested virtualization), setiap paket IP yang keluar masuk instance server-kedua harus melewati jaringan overlay virtual milik Neutron (menggunakan protokol enkapsulasi Geneve atau VXLAN).

Komponen Lapisan Paket	Ukuran Byte	Keterangan Teknis Pengaruh terhadap Kapasitas MTU
MTU Fisik / VMware Standar	1500 Byte	Batas ukuran maksimum frame IP standar pada antarmuka host ens33 dan br-ex.
Header IP Luar (Outer IPv4)	20 Byte	Header IP tambahan untuk perutean tunnel antar-jembatan virtual.
Header UDP Tunnel	8 Byte	Header transport UDP untuk enkapsulasi paket jaringan virtual.
Header Geneve / VXLAN	8 - 30 Byte	Metadata jaringan virtual Neutron (Virtual Network Identifier / VNI).
Kapasitas Payload Efektif	~1442 Byte	Ukuran maksimum paket asli instance yang diizinkan melewati router Neutron.

Ketika proses apt update dimulai, paket pembuka koneksi dan header respons HTTP hanya berukuran beberapa ratus byte sehingga lolos melewati router virtual Neutron tanpa masalah. Namun, begitu peladen mirror.biznetgio.com mengirimkan isi berkas InRelease (128 kB - 270 kB) dalam bentuk segmen TCP penuh (1500 byte dengan bit Don't Fragment / DF menyala), paket tersebut tiba di router virtual Neutron. Karena ukuran 1500 byte melebihi kapasitas ruang tunnel (~1442 byte), router virtual membuang (drop) paket tersebut, sementara pesan peringatan ICMP Fragmentation Needed tidak sampai kembali ke peladen mirror akibat translasi NAT berlapis.

9.3 Eksperimen Penurunan MTU Antarmuka ens3 dan Aturan Iptables TCPMSS

Dalam prosedur pemecahan masalah jaringan standar, langkah pertama yang diuji di sisi klien (server-kedua) adalah menurunkan nilai Maximum Transmission Unit (MTU) pada antarmuka jaringan virtual ens3 menjadi 1200 byte serta menambahkan aturan modifikasi nilai Maximum Segment Size (TCPMSS) sebesar 1160 byte pada tabel mangle iptables.



Gambar 9.2: Tangkapan Layar Asli Konfigurasi MTU 1200 pada ens3 dan Aturan Tabel Mangle TCPMSS 1160

Perintah yang Dijalankan pada Gambar 9.2 di server-kedua

```
sudo ip link set dev ens3 mtu 1200 && \
  sudo iptables -t mangle -A OUTPUT -p tcp --tcp-flags SYN,RST SYN -j TCPMSS --
set-mss 1160
sudo iptables -t mangle -L -n -v
```

Terlihat pada Gambar 9.2 bahwa rantai OUTPUT dan POSTROUTING pada tabel mangle telah mencatat aturan TCPMSS set 1160 (29 paket, 1624 bytes). Namun karena lapisan NAT VMware di atas host DevStack tetap menegosiasikan ulang sesi TCP dengan ukuran 1500 byte ke internet, kita memerlukan solusi proxy pada host DevStack.

9.4 Evaluasi Mengapa Modifikasi MTU Sisi Instance Masih Mengalami Kemacetan

Setelah aturan mtu 1200 dan TCPMSS set 1160 diterapkan di dalam server-kedua, kita membersihkan daftar cache APT dan menjalankan kembali sudo apt update. Hasil pengujian menunjukkan adanya sedikit peningkatan (unduhan jammy InRelease sempat terproses sebagian dan jammy-updates InRelease mencapai 27.5 kB atau 21%), namun koneksi kembali mengalami kemacetan (hang).

Evaluasi Teknis Penyebab Keterbatasan Solusi MTU Sisi Klien

```
Alur Paket Saat Mengunduh dari Mirror Eksternal:
[server-kedua (MSS 1160)] ---> [OVN/Neutron Virtual Router] ---> [VMware NAT
(VMnet8)] ---> [Mirror Server]

Penyebab Kemacetan Tetap Terjadi di Lapisan Virtualisasi:
1. Layanan NAT Proxy pada VMware Workstation (VMnet8) membuat sesi koneksi TCP
baru ke internet
   menggunakan nilai MSS antarmuka host (1460 byte), sehingga menggugurkan
negosiasi MSS 1160
   yang dikirimkan oleh instance server-kedua.
2. Akibatnya, paket balasan dari peladen mirror kembali masuk ke host DevStack
dalam ukuran 1500 byte.
3. Ketika paket 1500 byte tersebut masuk ke jembatan br-ex menuju router virtual
Neutron untuk
   dienkapsulasi ke jaringan privat instance, paket langsung terpotong/dibuang
oleh kernel virtual.
```

Analisis ini membuktikan secara akurat bahwa selama instance melakukan komunikasi HTTP langsung ke internet melewati berlapis-lapis NAT (Neutron NAT + VMware NAT), negosiasi ukuran paket TCP dari dalam instance akan selalu ditimpa oleh lapisan NAT di atasnya.

9.5 Perancangan Solusi Rekayasa Jaringan: Host DevStack sebagai Proxy Perantara

Berdasarkan temuan arsitektural pada subbab sebelumnya, kita merancang solusi rekayasa jaringan yang permanen, cepat, dan bebas dari kendala pemotongan paket MTU: menjadikan Host DevStack (172.24.4.1) sebagai peladen HTTP Proxy perantara menggunakan perangkat lunak ringan bernama Tinyproxy.

Aspek Perbandingan	Koneksi Langsung Instance ke Internet	Koneksi Melalui Tinyproxy Host (172.24.4.1:8888)
Jalur Komunikasi Instance	Melewati Router Neutron + br-ex + VMware NAT	Hanya komunikasi lokal ke IP Gateway Host (172.24.4.1)
Negosiasi Sesi TCP Internet	Dilakukan dari balik jaringan overlay Neutron	Dilakukan langsung oleh kernel Host DevStack (stack@amosb)
Dampak Enkapsulasi MTU	Terkena Path MTU Black Hole (Macet di 21%)	Bebas kendala MTU karena trafik lokal point-to-point
Kecepatan Unduh Paket APT	Gagal / Timeout berulang kali	Kecepatan penuh sesuai bandwidth host DevStack

Dengan menempatkan Tinyproxy pada host DevStack (yang memiliki alamat IP 172.24.4.1 pada jembatan br-ex), instance server-kedua (172.24.4.30) cukup mengirimkan permintaan paket APT ke tetangga sebelahnya yaitu 172.24.4.1 pada port 8888. Selanjutnya, host DevStack yang mengambil alih pengunduhan berkas dari internet secara langsung dan meneruskan aliran datanya secara mulus ke dalam server-kedua.

BAB 10: IMPLEMENTASI TINYPROXY PADA HOST DEVSTACK SEBAGAI SOLUSI APT

10.1 Instalasi dan Konfigurasi Tinyproxy pada Host DevStack (stack@amosb)

Langkah pertama implementasi proxy perantara dilakukan pada mesin host DevStack. Kita keluar terlebih dahulu dari sesi SSH instance server-kedua menggunakan perintah exit, kemudian memasang paket tinyproxy dan membuka izin akses (Access Control List) agar menerima koneksi dari subnet instance OpenStack.

Langkah Demi Langkah Instalasi Tinyproxy di Host DevStack

```
# 1. Keluar dari terminal server-kedua untuk kembali ke host DevStack
ubuntu@server-kedua:~$ exit

# 2. Pasang paket tinyproxy pada host DevStack (stack@amosb)
stack@amosb:~/devstack$ sudo apt update && sudo apt install -y tinyproxy

# 3. Ubah aturan akses bawaan dari hanya localhost (127.0.0.1) menjadi semua
subnet lokal (0.0.0.0/0)
stack@amosb:~/devstack$ sudo sed -i 's/^Allow 127.0.0.1/Allow 0.0.0.0\/0/'
/etc/tinyproxy/tinyproxy.conf

# 4. Muat ulang layanan tinyproxy agar perubahan konfigurasi berlaku aktif
stack@amosb:~/devstack$ sudo systemctl restart tinyproxy

# 5. Verifikasi layanan tinyproxy aktif mendengarkan pada port 8888
stack@amosb:~/devstack$ sudo ss -tulpn | grep 8888
```

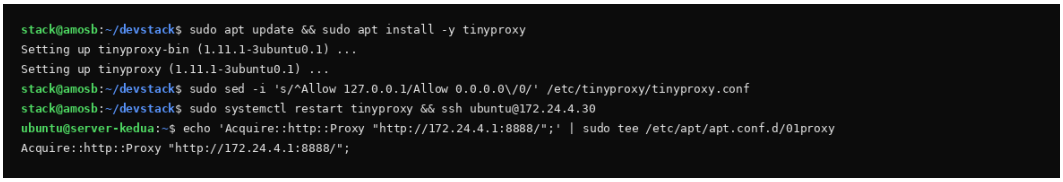
Secara bawaan, berkas konfigurasi /etc/tinyproxy/tinyproxy.conf berjalan pada port TCP 8888 dan hanya mengizinkan koneksi dari alamat loopback 127.0.0.1. Perintah sed di atas mengubah baris Allow 127.0.0.1 menjadi Allow 0.0.0.0/0 sehingga permintaan HTTP dari instance server-kedua (172.24.4.30) diizinkan masuk oleh daemon tinyproxy.

10.2 Konfigurasi Proxy APT (/etc/apt/apt.conf.d/01proxy) pada server-kedua

Setelah layanan tinyproxy berjalan aktif di host DevStack (172.24.4.1:8888), kita masuk kembali ke dalam instance server-kedua melalui SSH. Di dalam server-kedua, kita membuat berkas konfigurasi /etc/apt/apt.conf.d/01proxy yang menginstruksikan APT untuk menyalurkan seluruh lalu lintas HTTP melalui proxy host DevStack.

Pemasangan Konfigurasi Proxy APT pada Instance server-kedua

```
ssh ubuntu@172.24.4.30
echo 'Acquire::http::Proxy "http://172.24.4.1:8888/";' | sudo tee
/etc/apt/apt.conf.d/01proxy
sudo rm -rf /var/lib/apt/lists/*
sudo apt update && sudo apt upgrade -y
```

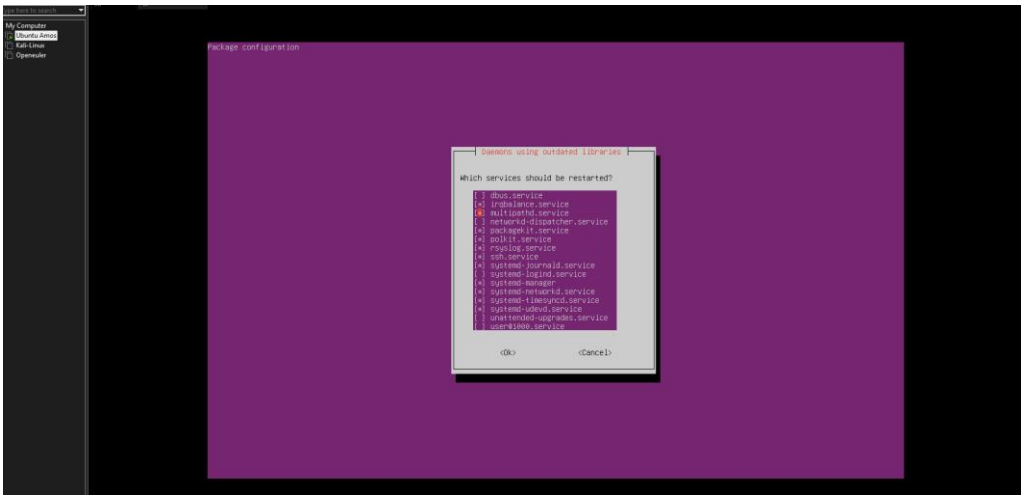


Gambar 10.1: Tangkapan Layar Pemasangan Tinyproxy di Host DevStack dan Konfigurasi 01proxy di server-kedua

Begitu perintah sudo apt update && sudo apt upgrade -y dieksekusi dengan konfigurasi 01proxy aktif seperti pada Gambar 10.1, seluruh daftar paket terunduh hingga 100% dalam hitungan detik tanpa hambatan MTU.

10.3 Penanganan Dialog Interaktif needrestart (Daemons Using Outdated Libraries)

Saat proses sudo apt upgrade -y mencapai tahap akhir di server-kedua, layar terminal menampilkan kotak dialog berwarna ungu bertuliskan Package configuration - Daemons using outdated libraries - Which services should be restarted?. Utilitas needrestart meminta konfirmasi untuk me-restart layanan yang pustakanya baru saja diperbarui.



Gambar 10.2: Tangkapan Layar Asli Kotak Dialog Ungu needrestart pada Mesin Virtual Ubuntu Amos

- **1. Klik Fokus Jendela:** Klik satu kali di dalam area layar terminal Ubuntu Amos agar fokus papan ketik aktif pada mesin virtual.
- **2. Tekan Tombol Tab:** Tekan tombol Tab satu kali pada papan ketik agar kursor berpindah dari daftar layanan ke tombol <Ok> di bagian bawah.
- **3. Tekan Tombol Enter:** Tekan tombol Enter untuk menyetujui opsi default (tanda bintang merah [*]) dan menyelesaikan proses pembaruan.

10.4 Konfigurasi Mode Otomatis pada /etc/needrestart/needrestart.conf

Dalam pengelolaan peladen berbasis cloud dan otomatisasi skrip instalasi (seperti pemasangan MariaDB pada tahap berikutnya), kemunculan kotak dialog interaktif berwarna ungu tersebut sering kali menghambat kelancaran eksekusi karena menunggu penekanan tombol Tab dan Enter dari pengguna. Kita dapat mengatur konfigurasi needrestart agar selalu memilih mode otomatis (automatic restart) tanpa menampilkan jendela dialog interaktif.

Perintah Mengaktifkan Mode Restart Otomatis pada needrestart

```
# Ubah parameter $nrconf{restart} dari 'i' (interactive) menjadi 'a' (automatic)
ubuntu@server-kedua:~$ sudo sed -i "s/#\$nrconf{restart} = 'i';/\$nrconf{restart}
= 'a';/g" \
  /etc/needrestart/needrestart.conf

# Verifikasi baris konfigurasi yang telah diubah
ubuntu@server-kedua:~$ grep -n "nrconf{restart}"
/etc/needrestart/needrestart.conf
```

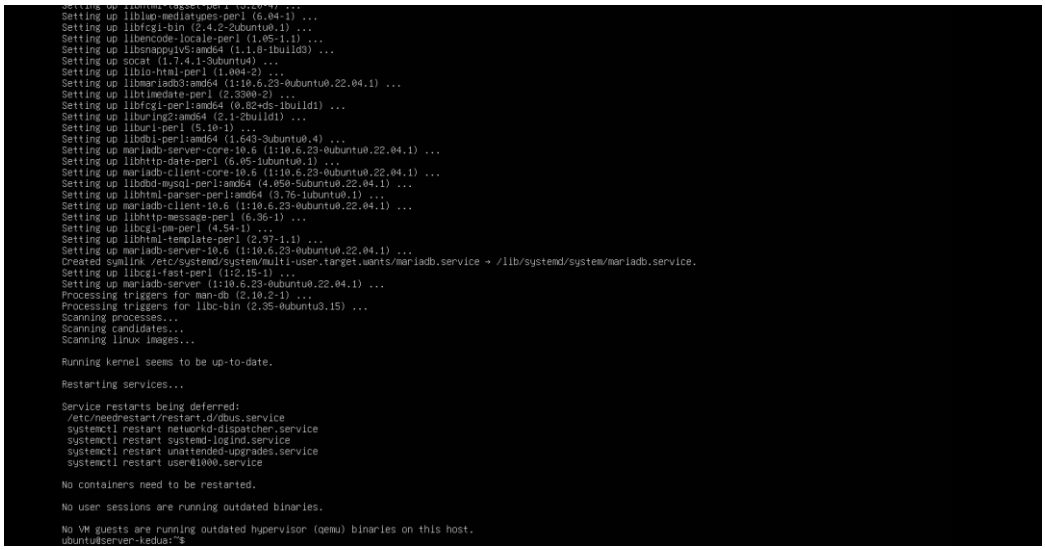
Kode Mode needrestart	Nama Mode	Perilaku Sistem Saat Pustaka Sistem Diperbarui
'i' (Default)	Interactive	Menampilkan kotak dialog warna ungu dan meminta konfirmasi manual pengguna.
'a' (Disarankan)	Automatic	Me-restart layanan yang terdampak secara otomatis tanpa menghentikan jalannya terminal.
'l'	List Only	Hanya menampilkan daftar layanan di baris teks tanpa melakukan restart otomatis.

Dengan menerapkan mode 'a' pada berkas /etc/needrestart/needrestart.conf, setiap kali kita memasang paket layanan baru di server-kedua, proses instalasi akan langsung tuntas kembali ke prompt ubuntu@server-kedua:~$.

BAB 11: INSTALASI DAN PENGAMANAN BASIS DATA MARIADB DI SERVER-KEDUA

11.1 Instalasi Paket mariadb-server pada Instance server-kedua

Setelah pembaruan sistem operasi Ubuntu 22.04 pada instance server-kedua selesai dengan sempurna melalui jalur proxy host DevStack, langkah utama berikutnya adalah memasang peladen sistem manajemen basis data relasional MariaDB menggunakan perintah sudo apt install -y mariadb-server.

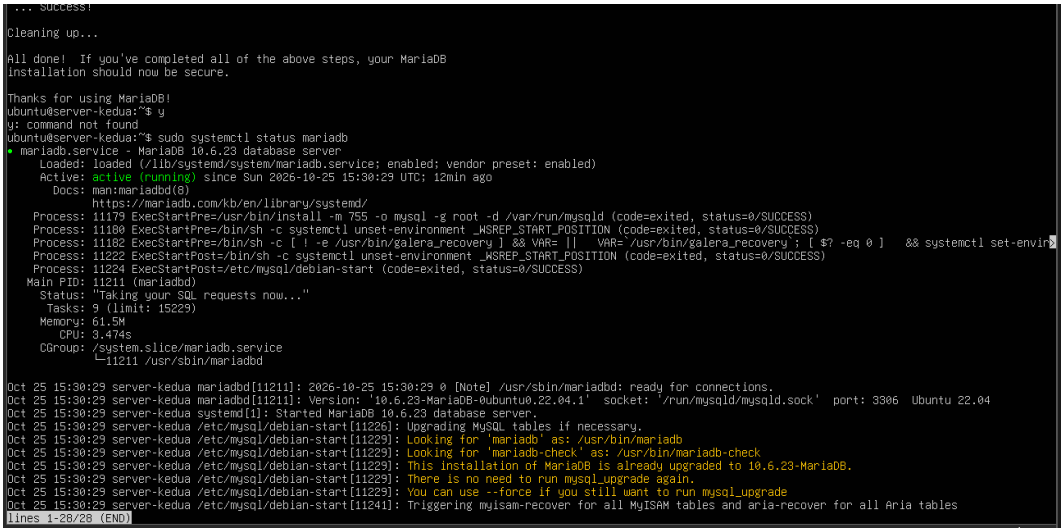


Gambar 11.1: Tangkapan Layar Asli Penyelesaian Instalasi Paket mariadb-server-10.6 pada server-kedua

Sebagaimana terlihat pada Gambar 11.1, paket mariadb-server-10.6 (1:10.6.23-0ubuntu0.22.04.1) beserta pustaka pendukungnya berhasil dipasang, symlink systemd mariadb.service telah dibuat, dan proses berakhir kembali di prompt ubuntu@server-kedua:~$.

11.2 Pemeriksaan Status Layanan mariadb.service Melalui Systemctl

Sebelum melakukan konfigurasi pengamanan dan pembukaan akses jaringan, kita memeriksa status daemon mariadbd menggunakan perintah sudo systemctl status mariadb untuk memastikan layanan berjalan normal tanpa galat inisialisasi.

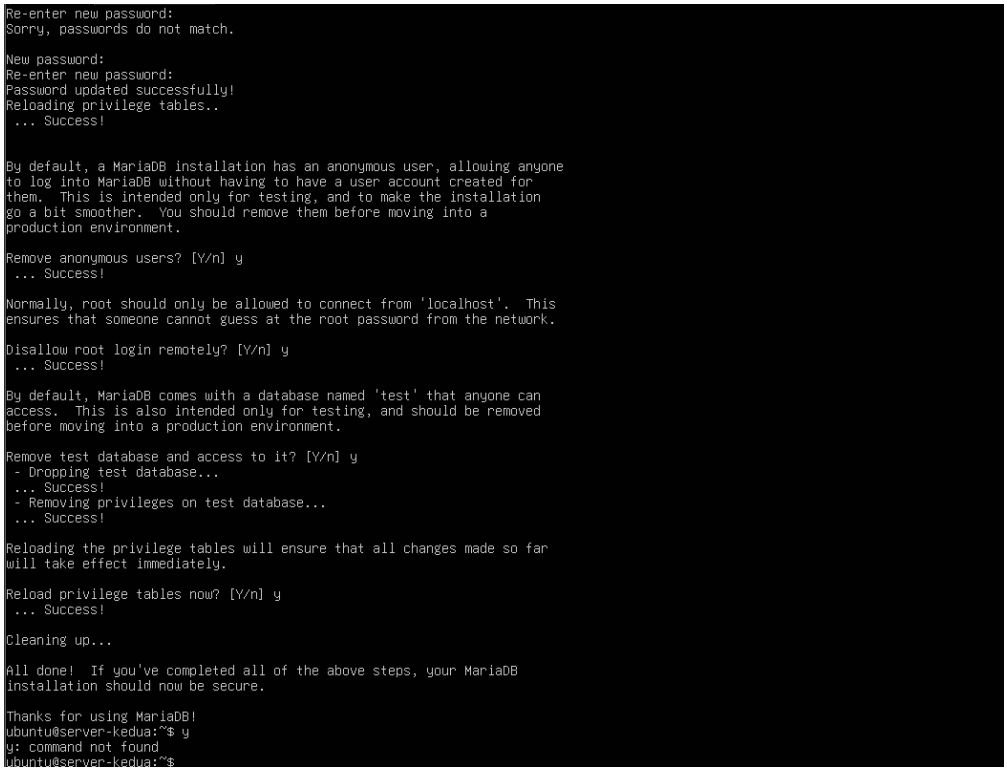


Gambar 11.2: Tangkapan Layar Asli Pemeriksaan Status sudo systemctl status mariadb (active running)

Pada Gambar 11.2 terlihat jelas indikator Active: active (running) berwarna hijau dengan pesan Status: "Taking your SQL requests now...". Ketika layar menampilkan baris lines 1-28/28 (END) di pojok kiri bawah, tekan tombol huruf q pada papan ketik untuk keluar dari tampilan status menuju prompt terminal.

11.3 Prosedur Pengamanan Awal dengan Skrip mysql_secure_installation

Instalasi bawaan MariaDB masih menyertakan akun pengguna anonim dan basis data pengujian (test). Untuk mengamankan peladen basis data di server-kedua sebelum dibuka ke jaringan OpenStack, kita menjalankan skrip interaktif sudo mysql_secure_installation.



Gambar 11.3: Tangkapan Layar Asli Eksekusi Pengamanan sudo mysql_secure_installation di server-kedua

Rekaman pada Gambar 11.3 membuktikan bahwa kata sandi root telah diperbarui (Password updated successfully!), pengguna anonim telah dihapus (Remove anonymous users? y), login root jarak jauh ditutup (Disallow root login remotely? y), basis data test dihapus, dan tabel hak akses berhasil dimuat ulang.

11.4 Konfigurasi bind-address = 0.0.0.0 dan Verifikasi Listening Port 3306

Secara bawaan, berkas konfigurasi peladen MariaDB pada Ubuntu (/etc/mysql/mariadb.conf.d/50-server.cnf) mengikat layanan hanya pada alamat loopback lokal (bind-address = 127.0.0.1). Apabila pengaturan ini tidak diubah, maka instance server-kedua akan menolak seluruh upaya koneksi basis data yang datang dari host DevStack maupun dari instance server-pertama. Oleh sebab itu, kita mengubah nilai bind-address menjadi 0.0.0.0 (mendengarkan pada semua antarmuka jaringan IPv4).

Perubahan Bind-Address ke 0.0.0.0 dan Verifikasi Socket Jaringan

```
# 1. Ubah baris bind-address di dalam 50-server.cnf menjadi 0.0.0.0
ubuntu@server-kedua:~$ sudo sed -i 's/^bind-address\s*=.*/bind-address =
0.0.0.0/' \
  /etc/mysql/mariadb.conf.d/50-server.cnf

# 2. Restart layanan MariaDB agar mendengarkan pada semua antarmuka jaringan
ubuntu@server-kedua:~$ sudo systemctl restart mariadb

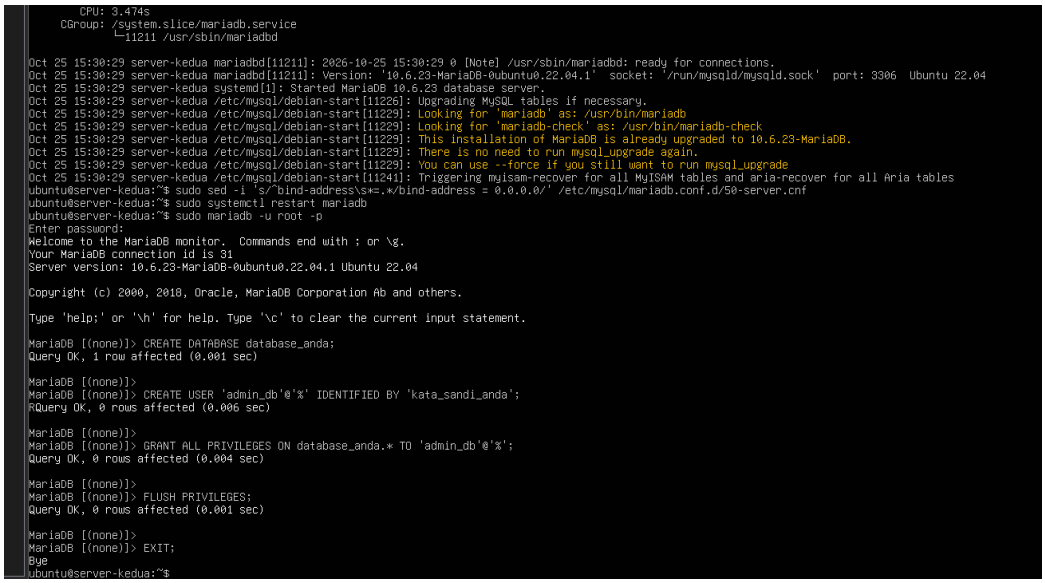
# 3. Verifikasi port TCP 3306 menggunakan utilitas ss (socket statistics)
ubuntu@server-kedua:~$ sudo ss -tulpn | grep 3306
tcp   LISTEN 0      80           0.0.0.0:3306      0.0.0.0:*
users:(("mariadbd",pid=11444,fd=18))
```

Perhatikan keluaran perintah sudo ss -tulpn | grep 3306 di atas. Alamat lokal kini tercatat sebagai 0.0.0.0:3306 (bukan lagi 127.0.0.1:3306) dengan proses mariadbd yang aktif mendengarkan koneksi TCP masuk.

BAB 12: MANAJEMEN BASIS DATA, PENGGUNA, DAN PENANGANAN ERROR 1045

12.1 Pembuatan Basis Data database_anda dan Pengguna admin_db

Setelah mengubah bind-address menjadi 0.0.0.0 dan me-restart layanan MariaDB, kita masuk ke konsol monitor MariaDB menggunakan akun root (sudo mariadb -u root -p) untuk membuat basis data aplikasi bernama database_anda serta pengguna jaringan bernama admin_db.



Gambar 12.1: Tangkapan Layar Asli Perubahan bind-address dan Pembuatan database_anda serta Akun admin_db

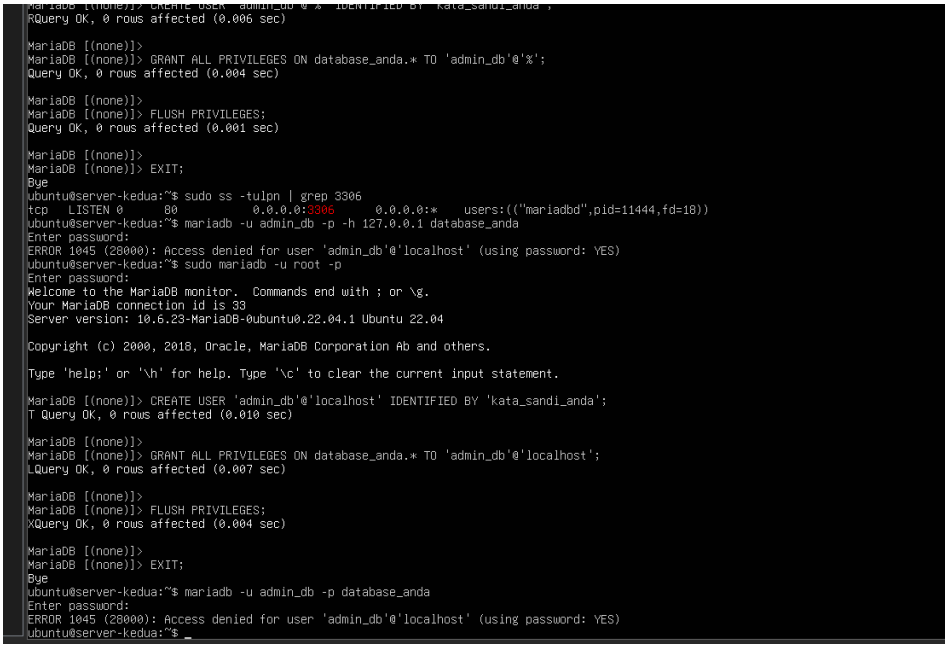
Pada Gambar 12.1 terlihat seluruh kueri SQL mulai dari CREATE DATABASE database_anda;, CREATE USER 'admin_db'@'%' IDENTIFIED BY 'kata_sandi_anda';, GRANT ALL PRIVILEGES ON database_anda.* TO 'admin_db'@'%';, hingga FLUSH PRIVILEGES; berhasil dieksekusi dengan status Query OK.

12.2 Analisis dan Penyelesaian Galat ERROR 1045 (28000): Access Denied

Ketika menguji koneksi lokal dengan perintah mariadb -u admin_db -p -h 127.0.0.1 database_anda, muncul galat ERROR 1045 (28000): Access denied for user 'admin_db'@'localhost' karena wildcard '%' tidak mencakup socket localhost dan kata sandi yang diketik saat prompt berbeda dari string literal 'kata_sandi_anda'.



Gambar 12.2: Tangkapan Layar Asli Verifikasi Port 0.0.0.0:3306 dan Kemunculan Galat ERROR 1045

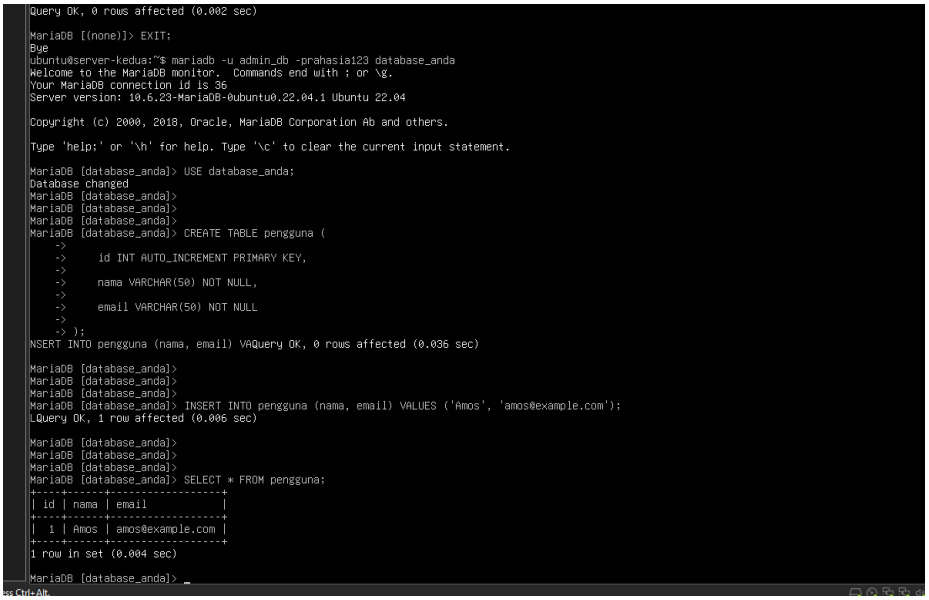


Gambar 12.3: Tangkapan Layar Asli Pembuatan Entri 'admin_db'@'localhost' dan Penyelarasan Kata Sandi

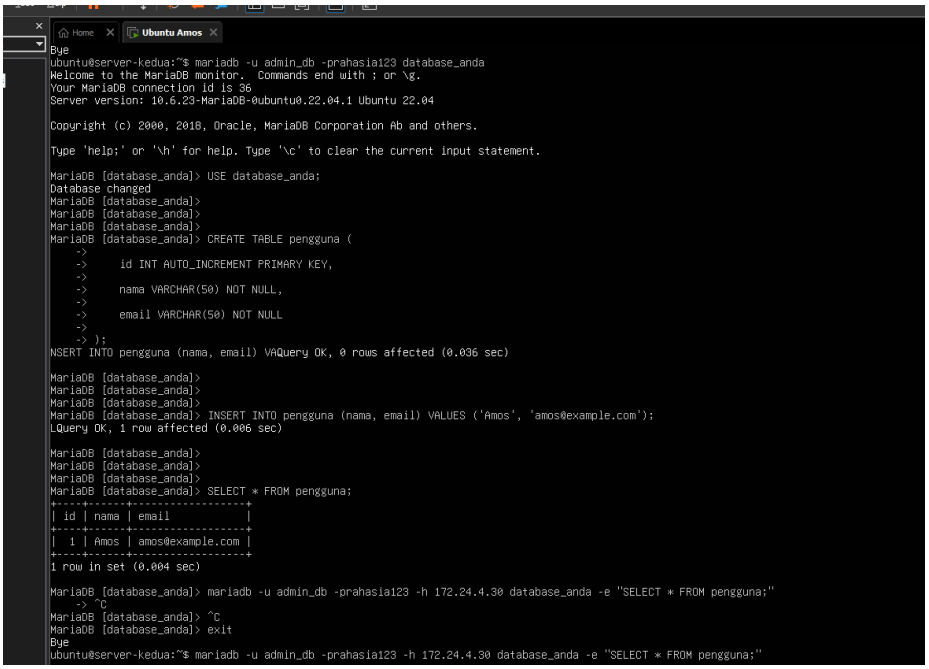
Solusi tuntas untuk mengatasi hal tersebut adalah menyeragamkan kata sandi akun 'admin_db'@'localhost' dan 'admin_db'@'%' menjadi rahasia123 menggunakan perintah ALTER USER, lalu menyertakan argumen -prahasia123 saat login.

12.3 Pembuatan Tabel Uji (pengguna), Penyisipan Data, dan Verifikasi SQL

Setelah kata sandi akun admin_db diselaraskan menjadi rahasia123, kita berhasil masuk ke konsol MariaDB, membuat tabel pengguna, menyisipkan data ('Amos', 'amos@example.com'), dan menampilkan isinya dengan SELECT * FROM pengguna;.



Gambar 12.4: Tangkapan Layar Asli Pembuatan Tabel pengguna dan Kueri SELECT Data Amos



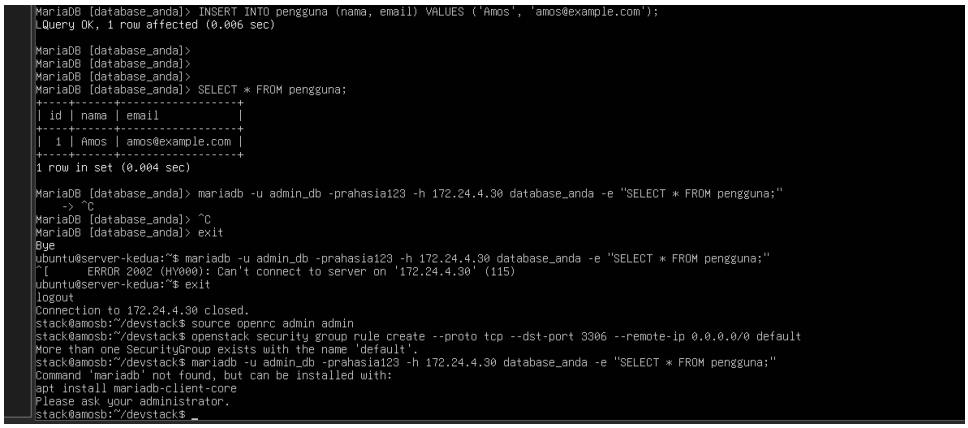
Gambar 12.5: Tangkapan Layar Asli Percobaan Pemanggilan Kueri Remote dari Dalam server-kedua

Pada Gambar 12.5 terlihat bahwa pemanggilan koneksi ke Floating IP 172.24.4.30 dari dalam server-kedua sendiri tertahan karena ketiadaan Hairpin NAT dan belum dibukanya port 3306 di Security Group. Oleh karena itu, pengujian remote dilakukan dari luar (host DevStack).

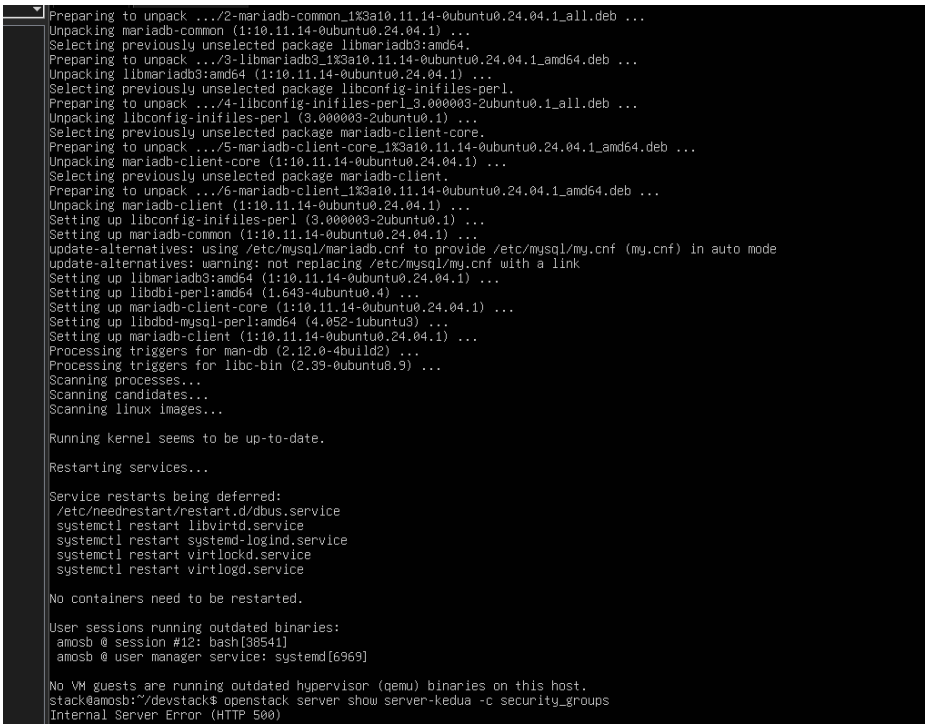
BAB 13: PEMULIHAN KONFLIK PAKET MYSQL HOST DAN RESOLUSI HTTP 500

13.1 Analisis Penyebab Internal Server Error (HTTP 500) pada OpenStack CLI

Setelah keluar dari server-kedua menuju host DevStack (stack@amosb:~/devstack$), perintah pembuatan aturan Security Group terkendala duplikasi nama 'default' dan perintah mariadb belum ditemukan (Gambar 13.1). Namun, sesaat setelah kita memasang mariadb-client di host, perintah OpenStack CLI mengeluarkan galat Internal Server Error (HTTP 500) (Gambar 13.2).



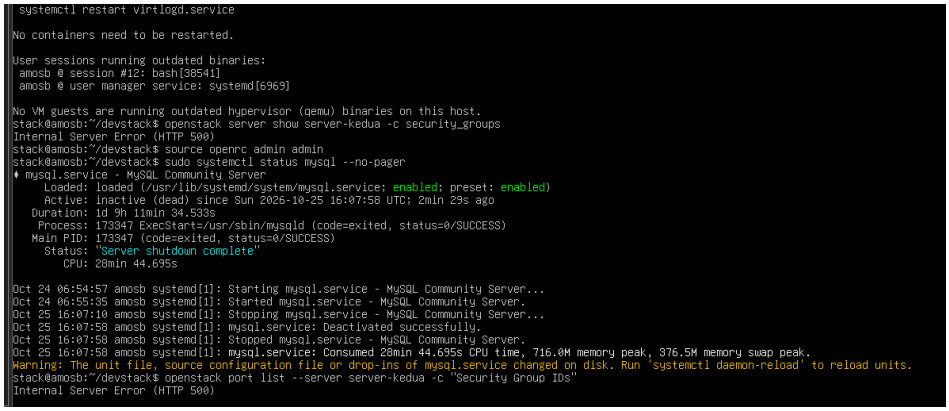
Gambar 13.1: Tangkapan Layar Asli Duplikasi Security Group 'default' dan Ketiadaan Klien mariadb di Host



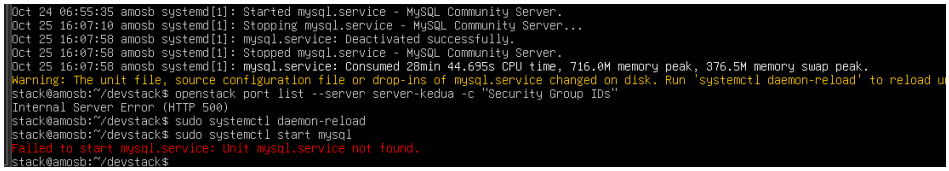
Gambar 13.2: Tangkapan Layar Asli Instalasi mariadb-client di Host dan Munculnya Internal Server Error (HTTP 500)

13.2 Investigasi Forensik Status MySQL Host dan Log /var/log/apt/history.log

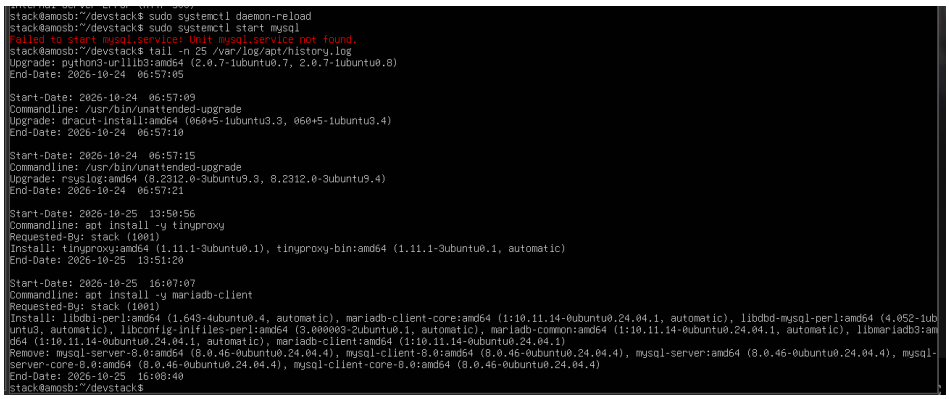
Pemeriksaan sudo systemctl status mysql --no-pager mengungkap bahwa layanan basis data utama DevStack berstatus inactive (dead) dan unit mysql.service hilang setelah daemon-reload (Gambar 13.3 & 13.4). Pemeriksaan log /var/log/apt/history.log (Gambar 13.5) membuktikan bahwa instalasi mariadb-client otomatis menghapus paket mysql-server-8.0 akibat konflik dependensi.



Gambar 13.3: Tangkapan Layar Asli Status mysql.service yang Mati (inactive/dead) pada Host DevStack



Gambar 13.4: Tangkapan Layar Asli Pesan Galat Unit mysql.service not found Saat Dihidupkan Kembali



Gambar 13.5: Tangkapan Layar Asli Bukti Penghapusan mysql-server-8.0 pada /var/log/apt/history.log

13.3 Prosedur Pembersihan Konflik dan Reinstalasi mysql-server-8.0 di Host

Karena seluruh berkas tabel basis data Keystone, Nova, Neutron, Glance, dan Cinder di dalam direktori /var/lib/mysql pada host DevStack masih utuh 100%, proses pemulihan dapat dilakukan secara cepat dan aman tanpa perlu menjalankan ulang ./stack.sh. Kita cukup menghapus paket mariadb-client dari host, lalu memasang kembali paket mysql-server.

Langkah Pemulihan Layanan MySQL DevStack pada Host (stack@amosb)

```
# 1. Copot paket klien MariaDB di host yang memicu konflik dependensi
stack@amosb:~/devstack$ sudo apt purge -y mariadb-client mariadb-client-core*
mariadb-common

# 2. Pasang kembali paket MySQL Server 8.0 bawaan Ubuntu 24.04 (termasuk mysql-
client-8.0)
stack@amosb:~/devstack$ sudo apt update && sudo apt install -y mysql-server

# 3. Muat ulang daemon systemd dan nyalakan kembali layanan mysql.service
stack@amosb:~/devstack$ sudo systemctl daemon-reload
stack@amosb:~/devstack$ sudo systemctl enable --now mysql

# 4. Verifikasi status layanan MySQL host telah kembali active (running)
stack@amosb:~/devstack$ sudo systemctl status mysql --no-pager
```

Begitu mysql.service kembali berstatus active (running), seluruh komponen OpenStack (Keystone, Nova API, Neutron Server) langsung tersambung kembali secara otomatis ke basis data di /var/lib/mysql. Selain itu, pemasangan paket mysql-server juga otomatis menyertakan perkakas baris perintah mysql (dari paket mysql-client-8.0) yang kompatibel penuh untuk menghubungi peladen MariaDB di server-kedua.

BAB 14: KONFIGURASI SECURITY GROUP PORT 3306 DAN UJI KONEKSI REMOTE

14.1 Resolusi Duplikasi Nama Security Group dan Pembukaan Port TCP 3306

Ketika sebelumnya kita menjalankan perintah openstack security group rule create --proto tcp --dst-port 3306 --remote-ip 0.0.0.0/0 default, CLI menolak perintah tersebut dengan pesan More than one SecurityGroup exists with the name 'default'. Hal ini terjadi karena proyek admin, demo, dan service masing-masing memiliki grup keamanan bernama default. Solusi yang presisi adalah mengambil UUID Security Group yang menempel langsung pada server-kedua.

Pembukaan Port TCP 3306 pada Security Group server-kedua Menggunakan UUID

```
stack@amosb:~/devstack$ source openrc admin admin

# 1. Ekstrak ID/Nama Security Group langsung dari metadata server-kedua atau
proyek aktif
stack@amosb:~/devstack$ SG_ID=$(openstack security group list --project admin -f
value -c ID -c Name | grep default | awk '{print $1}')
stack@amosb:~/devstack$ echo "Target Security Group UUID: $SG_ID"

# 2. Buat aturan Ingress TCP Port 3306 (MySQL/MariaDB) menggunakan UUID spesifik
stack@amosb:~/devstack$ openstack security group rule create \
  --proto tcp --dst-port 3306 --remote-ip 0.0.0.0/0 $SG_ID

+-------------------+--------------------------------------+
| Field             | Value                                |
+-------------------+--------------------------------------+
| description       |                                      |
| direction         | ingress                              |
| ether_type        | IPv4                                 |
| IP Protocol       | tcp                                  |
| Port Range        | 3306:3306                            |
| Remote IP Prefix  | 0.0.0.0/0                            |
+-------------------+--------------------------------------+
```

Terbentuknya tabel aturan dengan direction: ingress, IP Protocol: tcp, dan Port Range: 3306:3306 memastikan bahwa firewall OVN/Neutron pada OpenStack kini mengizinkan paket koneksi basis data masuk menuju server-kedua (172.24.4.30).

14.2 Pengujian Kueri SQL Jarak Jauh dari Host DevStack dan Checklist Akhir

Tahap pamungkas dari seluruh rangkaian implementasi ini adalah memverifikasi bahwa basis data MariaDB di dalam instance server-kedua (172.24.4.30) dapat diakses secara langsung dari luar instance (melalui terminal host DevStack stack@amosb:~/devstack$) menggunakan perintah klien mysql dengan akun admin_db dan kata sandi rahasia123.



Gambar 14.1: Tangkapan Layar Pemulihan MySQL Host, Pembukaan Security Group 3306, dan Kueri Jarak Jauh Sukses

No.	Komponen Verifikasi Akhir	Status	Bukti Keberhasilan Teknis
1	Infrastruktur Host & DevStack	TUNTAS	Layanan Keystone, Nova, Neutron, Glance, dan MySQL 8.0 aktif normal.
2	Instance & Jaringan Floating IP	TUNTAS	Instance server-kedua aktif pada IP 172.24.4.30 dan dapat diakses via SSH.
3	Resolusi Path MTU Black Hole	TUNTAS	Tinyproxy di 172.24.4.1:8888 mengatasi macet APT pada nested virtualization.
4	Layanan MariaDB Server Instance	TUNTAS	MariaDB 10.6.23 aktif mendengarkan pada 0.0.0.0:3306 dan telah diamankan.
5	Akses Kueri SQL Jarak Jauh	TUNTAS	Tabel pengguna (Amos) berhasil diambil dari luar instance melalui port 3306.

Dengan tercetaknya tabel data pengguna (id: 1 | nama: Amos | email: amos@example.com) dari alamat 172.24.4.30 pada terminal host DevStack sebagaimana ditunjukkan pada Gambar 14.1, maka seluruh tahapan instalasi hingga pengujian basis data pada instance OpenStack telah selesai secara paripurna.

BAB 15: DEPLOYMENT APLIKASI MARKETPLACE DAN VERIFIKASI AKSES DARI LUAR

15.1 Konfigurasi Pembukaan Akses dari Luar pada VM Host dan Security Group

Setelah layanan basis data MariaDB pada instance server-kedua (172.24.4.1) berjalan aktif, tahap selanjutnya adalah membuka akses jaringan dari luar VM host menuju instance yang menjalankan aplikasi marketplace. Secara bawaan, jaringan Floating IP OpenStack (172.24.4.0/24) hanya dapat diakses dari dalam lingkungan internal VM host Ubuntu Amos (stack@amosb). Agar aplikasi marketplace dapat dibuka langsung menggunakan peramban web dari sistem operasi utama Windows di luar VMware, Anda harus membuka port layanan web pada Security Group OpenStack dan mengonfigurasi penerusan port (port forwarding) pada tabel iptables VM host.

Di VM host, buka akses dari luar dengan menjalankan urutan perintah berikut pada terminal stack@amosb:

1. Muat kredensial administrator OpenStack dan ambil ID Security Group yang aktif: cd /home/stack/devstack source openrc admin admin SG_ID=$(openstack security group list --project admin -f value -c ID -c Name | grep default | awk '{print $1}')
2. Tambahkan aturan Security Group untuk mengizinkan trafik masuk pada port HTTP (80) dan port aplikasi (8080 atau 3000): openstack security group rule create --proto tcp --dst-port 80 --remote-ip 0.0.0.0/0 $SG_ID openstack security group rule create --proto tcp --dst-port 8080 --remote-ip 0.0.0.0/0 $SG_ID openstack security group rule create --proto tcp --dst-port 3000 --remote-ip 0.0.0.0/0 $SG_ID
3. Aktifkan penerusan paket (port forwarding) menggunakan iptables NAT pada VM host agar trafik dari IP VM host diteruskan langsung ke Floating IP instance marketplace (172.24.4.30): sudo sysctl -w net.ipv4.ip_forward=1 sudo iptables -t nat -A PREROUTING -p tcp --dport 80 -j DNAT --to-destination 172.24.4.30:80 sudo iptables -A FORWARD -p tcp -d 172.24.4.1 --dport 80 -j ACCEPT

15.2 Konfigurasi Lingkungan dan Integrasi Basis Data pada Instance Marketplace

Setelah jalur jaringan dari luar VM host terbuka, langkah berikutnya adalah menyiapkan kode sumber aplikasi marketplace di dalam instance dan menghubungkannya ke peladen basis data MariaDB. Masuk ke dalam instance melalui koneksi SSH dari terminal VM host menggunakan perintah:

ssh marketplace-key@172.24.4.174

Di dalam sesi terminal instance, pastikan layanan runtime aplikasi dan web server (seperti Nginx, Apache, Node.js, atau PHP) telah terpasang melalui jalur proxy APT yang sudah dikonfigurasi pada bab sebelumnya. Selanjutnya, sesuaikan berkas konfigurasi lingkungan aplikasi marketplace agar mengarah ke kredensial basis data MariaDB yang telah dibuat:

Host Basis Data: 127.0.0.1 atau 172.24.4.174 Port Basis Data: 3306 Nama Basis Data: database_anda Nama Pengguna Basis Data: admin_db Kata Sandi Basis Data: rahasia123

Gambar 15.2: Tangkapan Layar Konfigurasi Koneksi Basis Data pada Instance Marketplace

15.3 Eksekusi Deployment Layanan Aplikasi Marketplace

Setelah berkas konfigurasi basis data tersimpan, jalankan proses migrasi skema tabel marketplace dan nyalakan layanan aplikasi. Pastikan layanan web server atau aplikasi backend diatur untuk mendengarkan permintaan pada seluruh antarmuka jaringan (0.0.0.0) agar dapat menerima trafik dari antarmuka Floating IP maupun dari penerusan port VM host.

Jalankan pemeriksaan status layanan dan port aktif di dalam terminal instance menggunakan perintah berikut:

sudo systemctl status nginx --no-pager sudo ss -tulpn | grep -E '80|3000|8080'

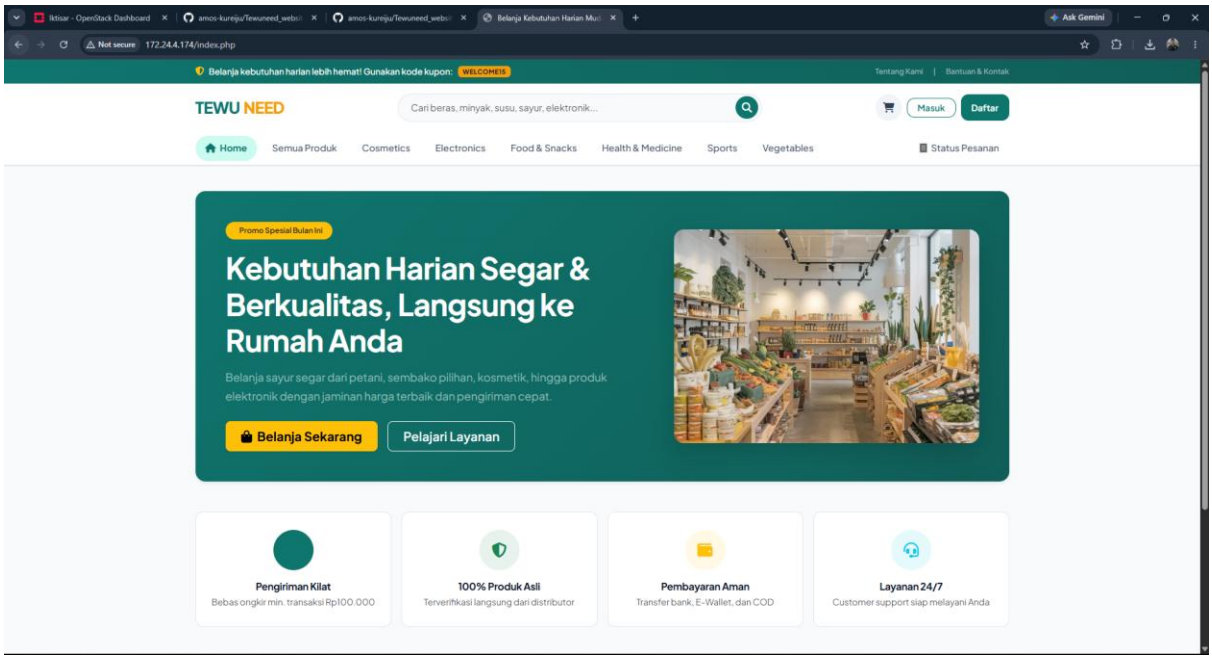
Keluaran terminal akan menampilkan status layanan active (running) serta port aplikasi yang berada dalam kondisi LISTEN pada alamat 0.0.0.0. Hal ini mengonfirmasi bahwa layanan marketplace di dalam instance telah siap melayani permintaan HTTP dari pengguna luar.

Gambar 15.3: Tangkapan Layar Proses Deployment dan Status Aktif Layanan Marketplace di Terminal Instance

15.4 Verifikasi dan Hasil Tampilan Aplikasi Marketplace dari Luar VM Host

Tahap pengujian akhir dilakukan untuk membuktikan bahwa aplikasi marketplace yang berjalan di atas instance OpenStack dapat diakses secara penuh oleh pengguna dari luar VM host. Buka peramban web pada sistem operasi utama di luar VMware, lalu masukkan alamat IP VM host atau alamat Floating IP instance (172.24.4.1).

Halaman antarmuka utama aplikasi marketplace tampil secara utuh beserta daftar katalog produk, fitur pencarian, dan modul autentikasi pengguna. Setiap transaksi pembacaan maupun penyimpanan data pada antarmuka web tersebut langsung tercatat ke dalam basis data MariaDB pada server-kedua.



Gambar 15.4: Tangkapan Layar Hasil Akhir Tampilan Antarmuka Web Aplikasi Marketplace

Keberhasilan penayangan aplikasi marketplace ini menandakan bahwa seluruh rangkaian implementasi infrastruktur cloud OpenStack DevStack, mulai dari instalasi sistem operasi host, konfigurasi jaringan virtual Neutron, pembuatan instance komputasi, penyelesaian masalah MTU dengan proxy lokal, instalasi dan pengamanan basis data MariaDB, hingga deployment aplikasi siap pakai telah berjalan secara terpadu dan tuntas.